2019 届研究生硕士学位论文

分类号：_____ 学校代码：10269

密　级：_____ 学　　号：51164500033



# 華東師範大學

East China Normal University

# 硕士学位论文

MASTER'S DISSERTATION

# 论文题目：云计算环境中利润最大化的个性用户服务请求调度研究

**院　　系：** 计算机科学与技术系

**专　　业：** 计算机应用技术

**研究方向：** 实时系统与云计算

**指导教师：** 魏同权　副教授

**学位申请人：** 徐过

2019 年 5 月 8 日

Dissertation for Master Degree in 2019

University Code: 10269
Student ID: 51164500033

# East China Normal University

## Title: Profit-driven Service Request Scheduling for Heterogeneous Users in Cloud Computing


Department: Department of Computer Science

Major: Computer Application and Technology

Research direction: Cloud Computing

Supervisor: Prof. Tongquan Wei

Candidate: Guo Xu


May 8, 2019

# 华东师范大学学位论文原创性声明

郑重声明：本人呈交的学位论文《云计算环境中利润最大化的个性用户服务请求调度研究》，是在华东师范大学攻读硕士/博士（请勾选）学位期间，在导师的指导下进行的研究工作及取得的研究成果。除文中已经注明引用的内容外，本论文不包含其他个人已经发表或撰写过的研究成果。对本文的研究做出重要贡献的个人和集体，均已在文中作了明确说明并表示谢意。

作者签名：徐诗　　日期：2019年 5月 10日

# 华东师范大学学位论文著作权使用声明

《云计算环境中利润最大化的个性用户服务请求调度研究》系本人在华东师范大学攻读学位期间在导师指导下完成的硕士/博士（请勾选）学位论文，本论文的著作权归本人所有。本人同意华东师范大学根据相关规定保留和使用此学位论文，并向主管部门和学校指定的相关机构送交学位论文的印刷版和电子版；允许学位论文进入华东师范大学图书馆及数据库被查阅、借阅；同意学校将学位论文加入全国博士、硕士学位论文共建单位数据库进行检索，将学位论文的标题和摘要汇编出版，采用影印、缩印或者其它方式合理复制学位论文。

本学位论文属于（请勾选）

（ ）1.经华东师范大学相关部门审查核定的"内部"或"涉密"学位论文*，于　年　月　日解密，解密后适用上述授权。

（√）2.不保密，适用上述授权。

导师签名　　本人签名徐诗

2019年5月10日

* "涉密"学位论文应是已经华东师范大学学位评定委员会办公室或保密委员会审定过的学位论文（需附获批的《华东师范大学研究生申请学位论文"涉密"审批表》方为有效），未经上述部门审定的学位论文均为公开学位论文。此声明栏不填写的，默认为公开学位论文，均适用上述授权）。

# 徐过 硕士学位论文答辩委员会成员名单

| 姓名 | 职称 | 单位 | 备注 |
| --- | --- | --- | --- |
| 沈富可 | 教授级高级工程师 | 华东师范大学 | 主席 |
| 沈建华 | 副教授 | 华东师范大学 | |
| 黄新力 | 副教授 | 华东师范大学 | |
| | | | |
| | | | |

# 摘要

随着企业和个人用户对计算资源的需求越来越大，云计算作为可以高效提供计算资源和服务的按需付费模式变得越来越受欢迎，云计算提供商也层出不穷。云计算提供商的典型目标是最大化其利润，用户的主要目标是获得自己满意的服务价格和服务质量，而且每个用户对于服务价格和服务质量都有自己的偏好。作为资源的提供者，云计算提供商应该了解不同个性的用户对服务价格和服务质量的不同偏好，并根据用户的个性化偏好进行服务请求的分配和调度以在用户满意的前提下最大化利润。

本文考虑了用户的个性化偏好，设计了新颖的实时服务请求调度算法，在用户满意度的约束条件下，优化了云计算提供商的利润。具体来说，

1. **本文提出了用户个性的概念，并实现了对不同个性用户满意度的精准建模。**本文建立了一个统计模型来估计用户对于给定服务价格和服务质量的满意度，该统计模型能够预测到不同个性的用户对于服务价格和服务质量的个性化需求，从而让用户得到满意的服务。
2. **本文设计了基于整数线性规划（Integer Linear Programming, ILP）的云提供商利润优化方案。**本文基于 ILP 对个性用户服务请求的分配和调度进行建模和求解，以获得用户满意度约束下云提供商的最优利润方案，并且提出了基于 ILP 的方法的不足。
3. **本文提出了基于价值评估和交叉熵的启发式利润优化算法。**为了克服基于 ILP 的方法的缺点，本文提出了一个启发式利润优化算法，用于个性用户服务请求的实时分配和调度。该算法可以得到用户服务请求的近似最优分配与执行顺序，从而实现用户满意度约束下云提供商的利润优化。

实验结果表明，本文提出的用户满意度模型能够准确预测出用户的满意度分数。相比于其他三个已有的服务请求调度方案，所提出的基于 ILP 的利润优化方案能够实现高达 97.06%、42.45%和 23.58%的平均利润提升，所提出的启发式利

润优化算法能够实现高达 84.76%、31.22%和 10.35%的平均利润提升。

**关键词**：云计算，个性用户，利润最大化，用户满意度，服务请求调度，价值评估，交叉熵

# ABSTRACT

As people's demand for computing resources grows, cloud computing becomes more and more popular as a pay-as-you-go model that can efficiently provide computing resources and services, and cloud computing providers are also emerging.The typical goal of cloud computing providers is to maximize their profits. The main goal of users is to obtain their own satisfactory service prices and quality of service, and each user has their own preferences for service prices and quality of service. As a provider of services, cloud computing providers should understand the personalized preferences of service prices and quality for users with different personalities, and assign and schedule service requests based on their personalized preferences to maximize profits under user satisfaction constraints.

This paper considers the user's personalized preferences, designs a novel real-time service request scheduling algorithm, and optimizes the profit of the cloud computing provider under the constraint of user satisfaction. Specifically,

1. This paper proposes the concept of user personality and achieves accurate modeling of the satisfaction of different personality users. This paper establishes a statistical model to estimate the user's satisfaction with the given service price and service quality. The statistical model can predict the personalized demand of service price and service quality for users with different personalities, so that users can get satisfactory service.

2. This paper designs a cloud provider profit optimization scheme based on Integer Linear Programming (ILP). In this paper, based on ILP, the allocation and scheduling of personalized user service requests are modeled and solved to obtain the optimal profit scheme of cloud providers under the constraint of user satisfaction, and the shortcomings of ILP-based methods are proposed.

3. This paper proposes a heuristic profit optimization algorithm based on value

evaluation and cross entropy. In order to overcome the shortcomings of the ILP-based method, this paper proposes a heuristic profit optimization algorithm for real-time allocation and scheduling of personalized user service requests. The algorithm can obtain the approximate optimal allocation and execution order of the user service request, so as to realize the profit optimization of the cloud provider under the constraint of user satisfaction.

Experimental results show that the user satisfaction model proposed in this paper can accurately predict the user satisfaction score. Compared with the other three existing service request scheduling schemes, the proposed ILP-based profit optimization scheme can achieve an average profit increase of up to 97.06%, 42.45% and 23.58%, and the proposed heuristic algorithm can achieve an average profit increase of up to 84.76%, 31.22% and 10.35%.

**Keywords:** Cloud Computing, Personalized Users, Profit Maximization, User Satisfaction, Service Request Scheduling, Value Evaluation, Cross Entropy

# 目录

**摘要** ..... **I**
**ABSTRACT** ..... **III**
**目录** ..... **V**
**插图目录** ..... **VII**
**表格目录** ..... **VIII**
**第一章 引言** ..... **1**
1.1 研究背景与意义 ..... 1
1.2 国内外研究现状分析 ..... 3
1.2.1 云定价策略研究 ..... 3
1.2.2 服务质量优化研究 ..... 5
1.2.3 服务器能耗优化研究 ..... 6
1.3 论文的研究目标和内容 ..... 8
1.4 论文的创新点 ..... 9
1.5 论文的组织架构 ..... 10
**第二章 相关模型和问题定义** ..... **12**
2.1 系统架构模型 ..... 12
2.2 用户模型 ..... 13
2.2.1 服务请求模型 ..... 13
2.2.2 用户满意度模型 ..... 14
2.3 云服务提供商模型 ..... 16
2.3.1 云服务平台 ..... 16
2.3.2 多服务器系统 ..... 17
2.3.3 收入模型 ..... 18
2.3.4 成本模型 ..... 19
2.4 问题定义 ..... 21
2.5 本章小结 ..... 21
**第三章 基于统计模型的用户满意度预测研究** ..... **22**
3.1 大五人格与测量方法 ..... 22
3.2 人格对于用户满意度的影响探究 ..... 24
3.3 以人格为导向的用户满意度建模 ..... 26
3.4 基于用户满意度模型确定用户服务请求属性 ..... 29
3.5 本章小结 ..... 30
**第四章 基于整数线性规划的云提供商利润优化研究** ..... **32**

4.1 多核服务器向虚拟单核服务器的转换 ..... 32
4.2 基于整数线性规划的利润优化研究 ..... 34
4.2.1 整数线性规划问题建模和求解 ..... 34
4.2.2 基于整数线性规划的利润优化算法 ..... 36
4.3 基于整数线性规划的利润优化方案的不足 ..... 37
4.4 本章小结 ..... 38
**第五章 基于价值评估和交叉熵的云提供商利润优化研究 ..... 39**
5.1 启发式利润优化算法的理论支持 ..... 39
5.1.1 服务请求分配对云提供商和用户的影响 ..... 39
5.1.2 服务请求执行顺序对云提供商和用户的影响 ..... 41
5.2 基于价值评估和交叉熵的启发式利润优化算法研究 ..... 42
5.2.1 价值评估 ..... 42
5.2.2 交叉熵 ..... 45
5.3 基于价值评估和交叉熵的启发式利润优化算法 ..... 49
5.4 本章小结 ..... 51
**第六章 实验结果与分析 ..... 52**
6.1 实验设置 ..... 52
6.1.1 用户满意度预测模型实验设置 ..... 52
6.1.2 用户满意度约束下的云提供商利润优化实验设置 ..... 53
6.2 实验结果与分析 ..... 54
6.2.1 用户满意度预测模型实验结果与分析 ..... 54
6.2.2 用户满意度约束下的云提供商利润优化实验结果与分析 ..... 57
6.3 本章小结 ..... 63
**第七章 总结与展望 ..... 65**
7.1 本文总结 ..... 65
7.2 对未来的展望 ..... 66
**参考文献 ..... 67**
**科研成果 ..... 73**
**致谢 ..... 74**

# 插图目录

图 1.1 研究框架.....8
图 2.1 云计算系统架构.....12
图 2.2 服务请求示例.....14
图 2.3 云服务平台.....17
图 3.1 用户服务满意度分数.....25
图 3.2 用户服务满意度预测模型.....26
图 4.1 多核服务器向虚拟单核服务器的转换过程.....33
图 5.1 两种服务请求分配方案.....40
图 5.2 两种服务请求执行顺序.....41
图 5.3 概率密度函数的更新过程.....49
图 6.1 交叉验证过程.....53
图 6.2 Ameren 实时电费.....54
图 6.3 交叉验证中真实和预测的用户满意度得分的混淆矩阵.....55
图 6.4 新参与者的测试结果.....56
图 6.5 新参与者中真实和预测的用户满意度得分的混淆矩阵.....57
图 6.6 R = 6时五种算法云提供商利润优化对比.....58
图 6.7 R = 7时五种算法云提供商利润优化对比.....59
图 6.8 R = 8时五种算法云提供商利润优化对比.....59
图 6.9 R = 9时五种算法云提供商利润优化对比.....59
图 6.10 四种算法运行时间比较.....62

# 表格目录

表 3.1 大五人格特质及其描述 ..... 23

表 3.2 TIPI 问卷 ..... 24

表 5.1 探究服务请求分配影响的服务请求集 ..... 39

表 5.2 两种服务请求分配方案结果 ..... 40

表 5.3 探究服务请求执行顺序影响的服务请求集 ..... 41

表 5.4 两种服务请求执行顺序结果 ..... 42

表 6.1 服务器配置参数 ..... 53

表 6.2 交叉验证结果 ..... 55

表 6.3 所提出算法利润提升率 ..... 61

表 6.4 四种算法平均运行时间 ..... 62

# 第一章 引言

## 1.1 研究背景与意义

云计算将硬件和软件资源虚拟化为统一资源池，并通过因特网以按需使用、按使用量付费的方式为用户提供所需的资源，用户可以用低廉的价格使用云提供商维护的基础设施、平台或软件等服务[1][2][3]。同时，随着科学技术的快速发展，企业和个人用户对于计算资源的需求越来越大，这使得个人电脑无法满足高速增加的需求，而搭建和维护一个庞大的服务器集群需要的高额成本却让人望而止步，因此通过云计算来部署所需的软件或服务的方式变得越来越受欢迎。而网络的普及和带宽的提升使得用户通过因特网访问云计算资源变得更加便利，这进一步促进了云计算的发展和云提供商数量的增长。美国权威调研机构 Gartner 近日发布的数据显示，2018 年全球云服务市场份额达到了 1864 亿美元，同比增加 21.4%，同时 Gartner 还预测了全球云服务市场将在 2019 年达到 2211 亿美元的份额，在 2020 年达到 2602 亿美元的份额[4]。面对越发庞大的市场份额，云提供商应该了解云计算的运行模式和经济性以最大化利润，从而在未来的市场份额中占有一席之地。

在云计算中，基础设施服务提供商、云服务提供商和用户这三层结构组成了典型的云计算环境[5]。其中，基础设施服务提供商搭建和维护服务器集群，并使用虚拟机技术将服务器集群池化为统一资源池，以按需使用、按使用量付费的方式提供软硬件资源；云服务提供商则从基础设施服务提供商处租赁所需的服务器资源并构建服务环境以向用户提供云服务；用户则选择能够提供所需服务的云提供商并发送服务请求，然后根据服务请求的返回结果缴纳服务费用[6]。在这个典型的云计算环境中，云提供商的目标自然是最大化其利润，而云计算作为一种商业模式，它的利润组成与所有其他商业模式一样，即收入和成本。

云计算提供商的收入主要取决于服务质量和服务价格。对于服务质量，云提供商不能无限制的推迟服务请求的响应时间，这会降低用户的满意度和服务质量

从而影响到用户对服务请求的缴费。而如果云提供商总是用最好的服务器去执行用户服务请求来降低响应时间，虽然得到了用户的满意，但这会增加云提供商的开销。因此，云提供商需要一个合理的方案来用最小的开销保证用户的服务质量，让用户得到满意的服务从而增加利润。对于服务价格，云提供商自然希望越高越好，但过高的价格同样会降低用户的满意度，这会导致云产品购买人数减少，从而影响了云提供商的收入。若是服务价格较低，能够提高用户的满意度从而增加云产品的购买人数，但价格的降低可能会造成云提供商的利润损失[7]。因此，合理的价格不但能够保证用户的满意度，增加云产品购买人数，还能让云提供商获得较高的收入。

对于云计算提供商的成本，主要包括服务器的租赁费和服务器运营需要的电费。其中服务器的租赁费与服务器配置相关，服务器运营需要的电费和服务器能耗相关。云提供商若优先使用最好的服务器去执行服务请求，则用户能够得到最高的满意度和服务质量，但这会增加服务器的能耗从而增加电费成本。若云提供商优先使用执行速度最慢的服务器，则能够减少能耗和电费，但会降低用户满意度从而影响收入。因此，合理的调度策略能够让云提供商减少能耗的同时保证用户的满意度，从而增加利润。

此外，不同个性的用户对于服务价格和服务质量的偏好是不同的，只有了解用户对于服务价格和服务质量的偏好，云提供商才能为每个用户进行定制性的服务，才能真正保证了用户的满意度并优化利润。例如，对于偏好服务质量的用户，云提供商可以用最好的服务器去执行该用户的服务请求并收取较高的费用；而对于偏好服务价格的用户，则可以用执行速度较慢的服务器去执行该用户的请求，与之对应的是较低的服务价格。这样不但满足了用户的个性化需求，提高了用户的满意度，还能够增加云提供商的利润。因此，云提供商需要准确捕捉不同用户对于服务价格和服务质量的偏好，从而为每个用户定制服务质量和服务价格，然后再用有效的调度程序完成个性用户的服务请求调度以提高利润。综上所述，如何准确捕捉个性用户对于服务价格和服务质量的偏好，并在保证用户满意度的前提下优化云提供商的利润具有很高的研究价值。

## 1.2 国内外研究现状分析

由上文可知，云计算提供商的利润主要由收入和成本两部分组成，因此国内外对于优化云提供商利润的研究大多集中在提高收入和减少成本上。在云计算中，影响云提供商收入的因素有很多，例如产品定价、服务质量、市场需求和服务器的地理位置等，其中最重要的就是云产品的定价和服务质量，它们直接影响着云提供商对用户的收费。至于云提供商的成本，主要包括服务器集群的搭建成本和运营成本两部分，搭建成本是云提供商租赁或购买服务器的成本，运营成本是云提供商执行用户请求时服务器运行产生的电费成本，即服务器能耗。由于云提供商所维护的服务器集群一般不会有大规模的变化，搭建成本基本是固定的，因此本文专注于云定价、服务质量优化和服务器能耗优化在国内外的研究现状。

### 1.2.1 云定价策略研究

云提供商自然希望以高服务价格来获取高额收入，但这会降低用户对云产品的满意度并导致未来需求的下降。而低服务价格虽然可以提高用户的满意度和未来的需求，但这可能会降低云服务提供商的收入。因此，选择合理的云产品定价策略对于云提供商来说至关重要，国内外对于云产品定价的研究主要分为固定定价（Fixed Pricing）和动态定价(Dynamic Pricing)两种。

固定定价指的是云产品和服务的价格固定，不会随着市场环境的变化而改变，是目前各大云计算提供商所采取的最主流的定价方法。根据云提供商的服务种类和实际运营情况，固定定价又细分为认购定价(Subscription Based Pricing)，使用量付费(Pay Per Use)和分级定价(Tiered Pricing)三种。其中认购定价是软件即服务层的主流定价方法，即用户与云提供商签订使用协议，以固定的价格在购买周期内使用云产品，这种定价策略被 Adobe、Salesforce、XCalibre 和 XDrive 等公司采用。使用量付费和分级定价被基础设施即服务层和平台即服务层广泛使用，如阿里云、Google、Amazon 和 Microsoft 等公司都同时使用这两种定价策略。其中使用量付费指的是用户根据选择的云服务按使用量付费，使用量的单位可以是

时间单位或者流量单位，分级定价指的是云提供商将云服务按照产品配置分为不同的收费，用户根据自己的需求选择合适的云服务并付费。这两种方法一般被同时使用，分级定价将云服务或者产品按照配置分为不同的价位，然后用户按照使用量来使用并支付。

固定定价策略在国内外的相关文献中也被广泛研究。Ghamkhari 等人[8]采用统一的定价策略，为所有服务请求设定固定价格；文献[2][9]则论述了为所有服务请求设定固定价格的策略不能适应服务的差异性，并提出服务价格根据服务时间及服务需求变化的定价方法；张等人[10]使用定期更改服务价格的定价策略。尽管固定定价的策略在不断完善，由于用户需求的动态和突然性，即使是定期改变服务价格也无法及时反映市场供需的剧烈波动。因此，云提供商需要能够及时反映市场需求的定价方法，也就是动态定价。

动态定价是指云产品和服务的价格会随着市场需求、地理位置以及云产品本身的价值等变化而随着动态变化。在动态定价中，云产品的价格可以由云提供商根据市场环境来决定，也可以由用户自由竞争来决定。文献[11][12][13]就是云提供商捕捉市场环境变化以决定云产品的价格。Mac 等人[11]提出了一种遗传算法在竞争激烈的云市场中进行产品定价，该算法使云产品的价格能够随着市场变化自适应的改变，并为云提供商提供长期有利的定价策略。Xu 等人[12]考虑到用户需求的随机性，设计了一个最优的动态定价策略，该策略不但能从现有用户中获取更多收入，还能增加未来的需求。Zhao 等人[13]设计了一种在线算法，用于在跨不同位置的数据中心中动态定价虚拟机（Virtual Machine，VM）资源，以最大化云提供商的利润。而一些文献[14][15][16]将云产品交给用户自由竞争，以拍卖的形式让用户自己对云产品进行定价购买。Zhang 等人[14]提出了一种基于平滑分析和随机减少的拍卖机制，用于在地理分布式云数据中心中进行动态 VM 定价。Jiang 等人[15]设计了一种新颖的竞价语言和一个在线云拍卖的框架去有效地销售云资源。Wang 等人[16]将云产品定价建模为多单元组合拍卖，并提出贪婪的分配机制来优化申报的估值之和。但是，云用户可能通过恶意出价或隐藏他们

对云资源的偏好来影响拍卖结果并获得不公平的优势。这些行为将破坏其他正常用户的拍卖体验，大大降低拍卖效率并阻碍用户的参与。

动态定价使得云产品或服务的价格能够随着市场环境的改变而动态变化，既能满足用户需求的随机性和突然性，还能让云提供商获得高额收益。但动态定价的复杂性让云提供商需要付出额外的成本去计算实时产品价格，而且动态定价导致的产品价格变化会引起部分用户的不满意（比如在用户以较高价格购买产品后，产品的价格降低会让用户感觉不公平）。本文建立了一个基于用户个性的用户满意度预测模型，能够准确捕捉每个用户对服务价格的敏感度，并得到让用户满意的服务价格。此外，本文提出的用户满意度预测模型的训练是离线的，线上的使用只需要云提供商很小的性能开销，既解决了用户满意度的问题，又没有产生额外成本的负担。

### 1.2.2 服务质量优化研究

通常，服务质量是指云提供商对用户的性能保证，例如响应时间、可靠性和性能故障的补救措施等[17]。如果没有满足用户的性能要求，云提供商需要向云用户支付罚款。例如，如果用户没有从云服务提供商那里得到及时有效的服务请求响应，则云服务提供商将因低质量的服务而受到惩罚，罚款金额取决于服务水平协议（Service-Level Agreement，SLA）中的条款。但是，如果及时处理了用户的服务请求，则用户将全额支付 SLA 指定的服务费。因此，云服务提供商可以通过保证服务质量来减少收入的损失以增加利润。

国内外对如何改善服务质量进行了广泛的研究。提高服务质量的最简单方案就是云提供商维护过多的服务器集群以提供过度资源供应，这样即使在用户需求的高峰期，云提供商也能保证用户的高服务质量。但是，该解决方案不但导致低用户需求期间资源的浪费，还大大增加了云提供商的成本。一个更有效的方法是将用户分级，在用户需求高峰期先为高级用户保证高服务质量，然后将剩余的资源以低廉的价格供低级用户使用[18][19]。尽管如此，这种方法会导致低级用户不可预测的请求延迟、拒绝、终止和价格波动[20]。Mei 等人[21]设计了一种结合

短期租赁和长期租赁的租赁方案，该方案首先根据到达时间将用户的请求分配给长期租用的服务器，当等待队列中的服务请求到达其截止日期时，云提供商再从基础设施提供商处租用临时服务器来处理该请求。这样虽然可以保证所有用户的服务质量，但是由于临时服务器的租金较高，云提供商的成本也会增加。

还有一些文献[22][23]认为多个云提供商应该联合组成联盟来应对用户的高峰需求，让处于低用户需求期间的云提供商将未使用的虚拟机（VM）共享到云联盟中，处于高用户需求期间的云提供商可以从云联盟中借用 VM 来提高用户的服务质量。文献[24][25][26][27]研究了云联盟中资源共享的几种策略。Lena 等人[26]提出了一种云联盟形成机制，它增加了云提供商的动态资源扩展能力以满足用户的需求。为了保证云联盟的公平性，该机制使用规范化的估计 Banzhaf 值来计算联盟中每个云提供商的利润。为了保证云联盟的稳定性，即加入联盟的云提供商不会脱离联盟，该机制保证了每个云提供商在联盟中的利润不低于单独行动或加入其他联盟所获得的利润。Nancy 等人[27]提出了一系列基于多阶段博弈的云资源共享策略，该策略通过向其他云提供商销售未使用的 VM 来最大化云提供商的利润。这些策略不但可以帮助云联盟中的服务提供商确定借出和共享云资源的时机和规模，还能够维护云联盟的公平性和稳定性。但是，这些策略会因借出或共享 VM 时的 VM 迁移而增加每个云提供商的额外能耗，而且这些解决方案中的云提供商在做出共享决策时没有考虑到他们之间的竞争关系。

此外，以上方案都没有考虑到云用户的个性，即不同用户对于服务质量的要求是不同的，这和用户本身的性格有关。因此，本文建立了一个统计模型来预测不同个性用户对于服务质量的个性化需求，并根据用户的个性化需求设计了调度程序来满足用户的服务质量要求。

### 1.2.3 服务器能耗优化研究

云服务提供商必须支付服务器运行所消耗的电力。一方面，云服务提供商需要分配足够数量的服务器以满足用户的服务质量要求。另一方面，他们试图最小化服务器消耗的能量以减少电费的成本。这对云服务提供商来说是一个巨大的挑

战，因为迅速增长的用户服务请求需要更多的服务器来保证服务质量，这会导致电力成本的急剧增加。

国内外已经提出了许多方法来减少服务器的能量消耗。Li 等人[28]通过研究 VM 的放置来最大限度地降低网络流量成本和物理机器成本。Cui 等人[29]通过优化 VM 的动态重分配来降低网络通信成本。良好的 VM 布局策略可以提高资源利用率并降低云数据中心的流量成本[30][31]，然而，这些工作假设 VM 之间的流量为已知的，这有点不切实际。Lee 等人[32]则认为处理器共享对于服务器能耗的减少特别重要，通过处理器共享和两个利润驱动的调度算法，提出了云中资源共享的模型。但是，所提出的调度算法增加了违背 SLA 的次数，从而降低了服务质量。

还有一些文献[33][34]通过近似计算来减少服务器能耗。He 等人[33]研究了如何在云中以较低的计算成本获得足够的精度结果，可以用 0.32％-46.17％的计算成本达到高达 99％的精度。Chen 等人[34]利用数据分析的需求弹性来减少云中数据中心的资源消耗。需求弹性允许工作用较少的资源运行，但代价是适度的性能损失。

尽管这些方法能够有效的降低服务器能耗，但是没有考虑到能耗优化造成的性能损失对云提供商收入的影响。就像传统的计算机系统性能和能量平衡一样，电费成本的减少和服务收入的提升很难同时得到满足，电费成本的减少通常会导致服务质量低而减少云提供商收入。

因此，本文通过在云提供商的服务收入和电费成本之间达到良好的均衡来最大化利润。此外，本文还考虑了用户对于服务价格和服务质量的个性需求，这会让用户请求的种类变得复杂，大大增加了调度难度。因此，本文设计了一个两阶段的启发式优化算法用于个性用户服务请求的实时分配和调度，该算法应用了交叉熵和价值评估来优化调度过程。

## 1.3 论文的研究目标和内容

本文的研究目标是在存在个性用户的云环境中最大化云提供商的利润，同时保证用户的满意度。也就是说本文的研究目标有两个：保证用户满意度和最大化云提供商的利润。针对保证用户满意度方面，本文主要研究了用户个性对于用户满意度的影响，即不同用户对服务质量和服务价格的偏好。针对最大化云提供商利润方面，本文主要研究了个性用户服务请求分配和调度对云提供商利润的影响。所以本文的研究目标可以更详细的总结为：在云环境中，设计一套实时调度算法，确定个性用户服务请求在多服务器系统上的分配，以及在每个服务器上的执行顺序，能够在保证用户满意度的前提下最大化云提供商的利润。本文的研究框架如图 1.1 所示。

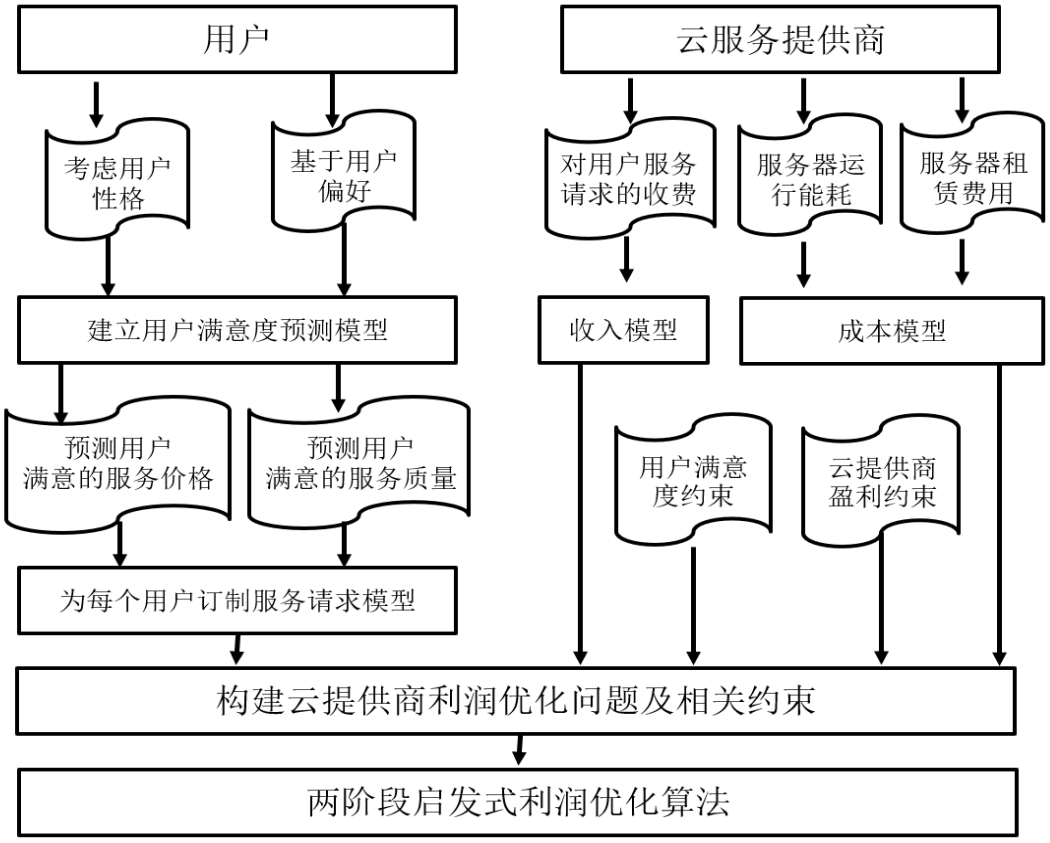



图 1.1 研究框架

如上图所示，本文综合考虑了用户的个性对满意度的影响以及云提供商的各项收入与成本。为了优化用户满意度，本文通过问卷调查的方式得到了用户对指定服务价格和服务质量的满意度以获得用户满意度预测模型。基于用户满意度预

测模型，本文可以得到让用户满意的价格和服务质量，并将此价格和服务质量作为用户的服务请求模型。然后，本文将所有用户的服务请求收费作为收入，将电费、租赁费作为成本，构建云提供商的利润模型。最后，本文设计了一个基于整数线性规划的云提供商最优利润调度方案，以及基于服务请求价值评估和交叉熵的启发式利润优化算法，解决了用户满意度约束下云提供商的利润优化问题。

## 1.4 论文的创新点

本文的主要创新为：

- 本文首先提出了用户个性的概念，即所有用户对于服务价格和服务质量的都有不同的偏好，这不但会影响到云提供商的利润，还会影响用户的满意度。基于用户个性，本文提出了一个新颖而全面的用户模型，该模型包括了用户服务请求模型和用户满意度模型。
- 然后，本文建立了一个统计模型来估计不同性格用户对于给定服务价格和服务质量的满意度，该统计模型能够得到用户对于服务价格和服务质量的偏好，从而为用户提供定制性服务以让用户得到满意的服务。
- 在个性用户模型下，用户的服务请求变得复杂多样，这让设计有效的调度程序具有了必要性。因此，本文基于整数线性规划 ILP 对个性用户服务请求的分配和调度进行了建模和求解，以获得用户满意度约束下云提供商的利润最大化求解方案。
- 为了减少调度算法给云提供商造成的额外负担，也为了提高调度算法的运行速度，本文提出了一个基于价值评估和交叉熵的启发式利润优化算法，用于个性用户服务请求的实时分配和调度。该启发式算法首先使用交叉熵将个性用户服务请求分配到各个服务器上，然后再使用经济学中的现值和机会成本对每个服务器上的用户请求进行价值估计，以决定用户请求在每个服务器上的执行顺序。通过启发式优化算法的这两个阶段，该算法可以得到每个服务请求的服务器位置和在服务器中的执行顺序，从而实现用户满意度约束下云提供商的利润优化。

## 1.5 论文的组织架构

本文一共有六章，具体的组织结构如下：

第一章首先介绍了云计算的相关背景知识和云提供商的利润组成，然后介绍了国内外对于云提供商利润优化的研究现状，并分析了相关研究的不足之处，接着介绍了本文的研究目标、研究内容和论文创新点，最后介绍了论文的具体结构安排。

第二章主要介绍了本文在研究中所采用的模型以及研究问题定义。模型主要包括系统架构模型、用户模型和云提供商模型。其中用户模型又包括了服务请求模型和用户满意度模型，云提供商模型包括了云服务平台、多服务器系统、收入模型和成本模型。基于这些模型，该章给出了研究问题的问题定义。

第三章建立了用户满意度预测模型去捕捉用户对于服务价格和服务质量的个性化需求。本部分主要包括大五人格的介绍以及测量方式、用户满意度模型的建模和求解预测模型参数的迭代算法设计。

第四章基于用户满意度预测模型，解决了云计算中用户满意度约束下的云提供商利润优化问题。本部分使用 ILP 对所研究的优化问题进行了建模和求解，提出了基于 ILP 的云提供商利润优化方案，该方案能够得到研究问题的最优解。接着，分析并讨论了基于线性规划的方案的局限性，提出了设计启发式利润优化算法的必要性。

第五章基于现值、机会成本和交叉熵设计了启发式利润优化算法来克服基于 ILP 的调度方案的局限性。该启发式算法首先使用交叉熵将个性用户的服务请求分配到各个服务器，然后使用经济学中的现值和机会成本对每个服务器中的服务请求进行价值评估，以决定服务请求在服务器中的执行顺序。通过这两个步骤，该算法可以得到服务请求在服务器中的位置和执行顺序，从而实现用户满意度约束下的云提供商利润优化。

第六章主要对用户满意度预测模型和用户满意度约束下的云提供商利润优化方案进行实验验证和结果分析。对于用户满意度预测模型，主要介绍了模型训练和模型验证的实验设置，验证了用户满意度预测的准确性；对于用户满意度约

束下的云提供商利润优化方案，主要介绍了实验中服务器、服务请求集合和实时电费的设置，验证了在不同用户满意度约束下利润和算法运行时间的优化效果，并与其他三种算法进行对比分析。

第七章总结了本文所做的工作并对未来工作进行展望。

# 第二章 相关模型和问题定义

本章主要介绍与研究问题相关的系统架构和模型，包括云计算系统架构、用户模型以及云提供商模型，其中用户模型又包括了服务请求模型和满意度模型，云提供商模型包括云服务平台模型、多服务器系统模型、收入模型和成本模型。基于这些模型，本章在最后给出了用户满意度约束下的云提供商利润优化问题的问题定义。

## 2.1 系统架构模型

如图 2.1 所示，在传统的云计算环境中一共包含了三个角色，基础设施服务提供商、云服务提供商和用户[5]。其中基础设施服务提供商负责维护服务器集群并通过虚拟化技术将服务器资源池化，以向云服务提供商按需的提供存储、计算和带宽等资源；云服务提供商从基础设施服务提供商处租赁虚拟机并根据自己的服务类型构建服务环境，以向用户提供各种云服务，如科学计算、图像处理、大数据分析等；用户则根据自己的需求选择对应的云服务提供商并发送服务请求，然后根据服务等级协议 SLA 和云服务提供商的返回结果缴纳费用[6]。

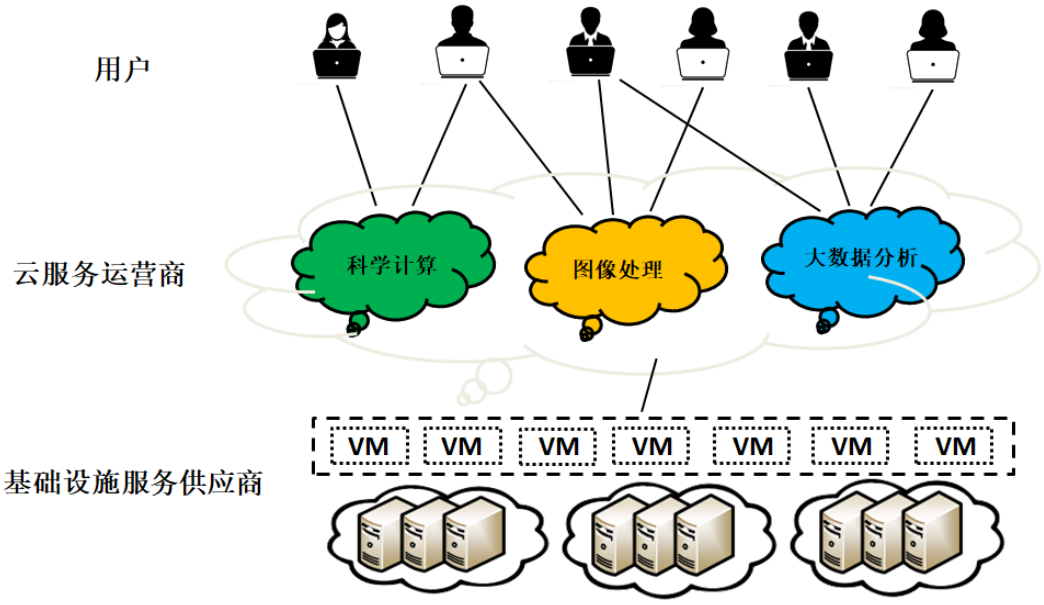



图 2.1 云计算系统架构

## 2.2 用户模型

在传统云计算环境中的三个角色中，用户根据自己的需求向云服务提供商提交请求，然后缴纳服务费用。服务费用由服务请求的指令条数、服务价格和服务质量共同决定。在本文中，每个用户都是个性的，用户的服务请求都是相互独立的，因此用户服务请求的价格、服务请求的指令条数和对服务质量的要求可能都是不同的。此外，每个用户对于服务价格和服务质量的偏好也是不同的，这将影响他们对于云服务的满意度。因此，本节从用户的服务请求和满意度两个方面对用户进行建模。

### 2.2.1 服务请求模型

由于用户的个性化，每个用户请求的属性也不尽相同，本文用一个五元组$\tau_m:\{W_m,p_m,D_m^{des},\alpha_m,D_m\}$来建模个性用户$m$的服务请求$\tau_m$，其中$W_m$是服务请求$\tau_m$的指令条数，$p_m$是用户$m$的服务价格，即每个指令条数的价格，$D_m^{des}$是用户希望的完成时间，在$D_m^{des}$之前完成即视为这次服务用户很满意，$\alpha_m$是用户对于低服务质量的容忍度，$D_m$为服务请求的截至日期，在$D_m$之后完成表示超过了该用户的容忍极限，视为满意度为 0，也就是完全没有用户体验。其中，指令条数$W_m$由服务请求决定，服务价格$p_m$、用户希望的完成时间$D_m^{des}$、低服务质量容忍度$\alpha_m$和服务请求截至日期$D_m$由用户的性格决定，本文将在后面的章节详细描述如何根据用户的性格确定这些属性的值。若$ft(\tau_m)$为服务请求$\tau_m$的实际完成时间，用户$m$的服务请求$\tau_m$应该缴纳的费用表示为$R(\tau_m,ft(\tau_m))$，则该用户的服务请求$\tau_m$的最终收费如下：

$$R(\tau_m,ft(\tau_m))=\begin{cases} W_m * p_m, & 0<ft(\tau_m)\le D_m^{des} \\ W_m * p_m-\frac{\left(ft(\tau_m)-D_m^{des}\right)}{\alpha_m}, & D_m^{des}<ft(\tau_m)\le D_m \\ 0, & ft(\tau_m)>D_m \end{cases} \tag{2.1}$$

该公式的一个实例如图 2.2 所示，该公式的详细意义为：

- 如果用户$m$的服务请求$\tau_m$在用户希望的完成时间$D_m^{des}$之前完成，即实际完

成时间$ft(\tau_m)$小于用户希望的完成时间$D_m^{des}$，则用户认为得到了高质量的服务，并对此次服务请求全额付款。$W_m * p_m$表示服务请求的指令条数和指令单价的乘积，即应付的全款金额。

- 如果用户$m$的服务请求$\tau_m$的实际完成时间$ft(\tau_m)$大于用户希望的完成时间$D_m^{des}$但是不大于服务请求的截至日期$D_m$，那么用户则认为得到了低服务质量，并且根据实际完成时间$ft(\tau_m)$决定最终付款金额。公式中$W_m * p_m$表示全款金额，$\frac{(ft(\tau_m)-D_m^{des})}{\alpha_m}$表示因低服务质量导致的惩罚金额，该惩罚金额由$ft(\tau_m)$、$D_m^{des}$和用户对于低服务质量的容忍度$\alpha_m$共同决定，则$W_m * p_m - \frac{(ft(\tau_m)-D_m^{des})}{\alpha_m}$为最终用户所需要支付的金额。选择$D_m$作为上界是因为$ft(\tau_m)$大于$D_m$时，用户的满意度降为0，将拒绝支付此次服务请求的费用，此次服务请求被视为请求超时。
- 如果用户$m$的服务请求$\tau_m$的实际完成时间$ft(\tau_m)$大于服务请求的截至日期$D_m$，则该用户认为此次服务请求延迟了过长时间，对此次服务极不满意，云提供商应该免费提供此服务，这也是云提供商严重违反 SLA 的惩罚。

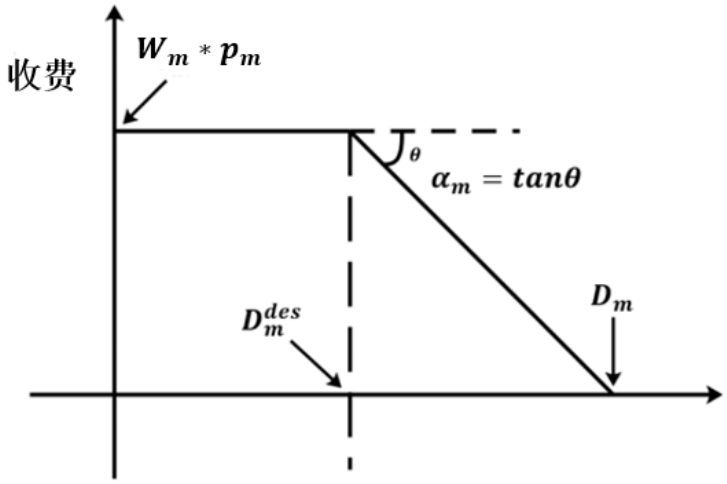



图 2.2 服务请求示例

### 2.2.2 用户满意度模型

随着科学技术的快速发展和对云服务需求的增加，市场上的云提供商也呈爆炸式增长，向所有不同个性的用户提供良好的服务体验可以让云提供商在竞争激

烈的云市场拥有更强的竞争力。

众所周知，从用户的角度来看，服务价格和服务质量是影响用户体验的两个重要因素，用户自然希望以低廉的价格得到质量最好的服务。但是从云提供商的角度来看，用户对于服务价格和服务质量的要求很难同时得到满足：更低的服务价格通常只能得到更差的服务质量，而改善服务质量将不可避免的提高服务价格。关于如何在服务价格和服务质量之间达到良好的权衡来提高用户满意度的研究已经有很多[6]，但是他们普遍缺乏对用户满意度的全面和准确的了解，忽略了用户的个性，而用户个性实际上在用户满意度中扮演着相当重要的角色，例如，一些用户可能想要低廉的价格，而另一些用户可能需要高质量的服务。本文首先从用户的角度对用户满意度进行了全新的定义，并在随后的章节提出了准确测量和管理用户满意度的方法。

本文首先引入一个潜变量 $p$ 来衡量用户个性，则潜变量 $p$ 可以表示为服务价格 $price$ 和服务质量 $qos$ 的加权求和，用户个性变量 $p$ 可表示为：

$$p = \gamma * qos + (1 - |\gamma|_1) * price \tag{2.2}$$

其中权重$\gamma = [\gamma_1, \gamma_2]$是一个测量用户对服务价格和服务质量偏好的向量，它满足$\gamma_1, \gamma_2 > 0$和$\gamma_1 + \gamma_2 < 1$。$|\gamma|_1$是$\gamma$的 L1 范数，其等于$\gamma_1 + \gamma_2$。在本文中，服务价格对应指令单价，服务质量对应服务请求的完成时间，也就是响应时间，因此参数$\gamma_1$和$\gamma_2$实际上分别测量用户对指令单价和响应时间的偏好。权重$\gamma$高（低）意味着用户更偏好高服务质量（低服务价格）。

由于用户满意度得分只是用户潜在兴趣的定量表达，本文使用线性模型基于用户潜在个性来表示用户满意度：

$$\mathcal{S} = \alpha_1 + \alpha_2 * p \tag{2.3}$$

其中，$\mathcal{S}$表示用户的服务满意度，$\alpha_1$和$\alpha_2$是两个参数，本文将在后续章节确定这两个参数的值。

## 2.3 云服务提供商模型

云服务提供商从基础设施提供商处租赁硬件资源并根据自己的服务类型搭建服务环境，然后向用户提供服务并根据服务质量收费。用户向云服务提供商提交他们的服务请求，接收云提供商返回的请求处理结果，并且基于服务的指令数量和质量来支付服务。在这个过程中，云提供商通过云服务平台来接收和调度用户服务请求，用多服务器系统来处理用户的服务请求。本文首先介绍云服务提供商使用的云服务平台和多服务器系统，然后对云提供服务商的收入和成本进行建模。

### 2.3.1 云服务平台

云服务平台是云提供商用来接收和调度用户服务请求的平台，用户提交的服务请求首先会被放置到云服务平台中的等候队列，然后云提供商根据自己的调度策略来处理这些服务请求。尽管在不同研究文献中云服务平台的构造细节不尽相同，但如图 2.3 所示的云服务平台被很多文献所使用[2][8][35][36]，该服务平台由服务请求调度队列、服务请求调度控制器和多个服务器工作节点组成。用户的服务请求首先被提交到云服务平台中的服务请求调度队列中等待调度控制器的调度，然后调度控制器检查各个服务器工作节点的工作状态并根据不同的调度策略分配用户服务请求。一些实际的云提供商（如 Amazon EC2、IBM Blue Cloud 和阿里云）就使用这种构造的云服务平台。在这些云提供商使用的云服务平台中，用户的服务请求被提交到服务请求调度队列后，由任务管理系统如 PBS(Portable Batch System)、SGE(Sun Grid Engine)和 Condor 进行集中分配。例如 Condor 是计算密集型服务请求专用的管理系统，它提供了服务请求排队机制、优先级机制和调度机制等资源管理机制。用户请求被提交到云服务平台后，Condor 负责将这些服务请求放入调度队列并决定它们在何时何地运行[37][38]。因此，本文将云服务平台建模为由服务请求等待队列、调度控制器和多服务器系统组成的平台是合理的。

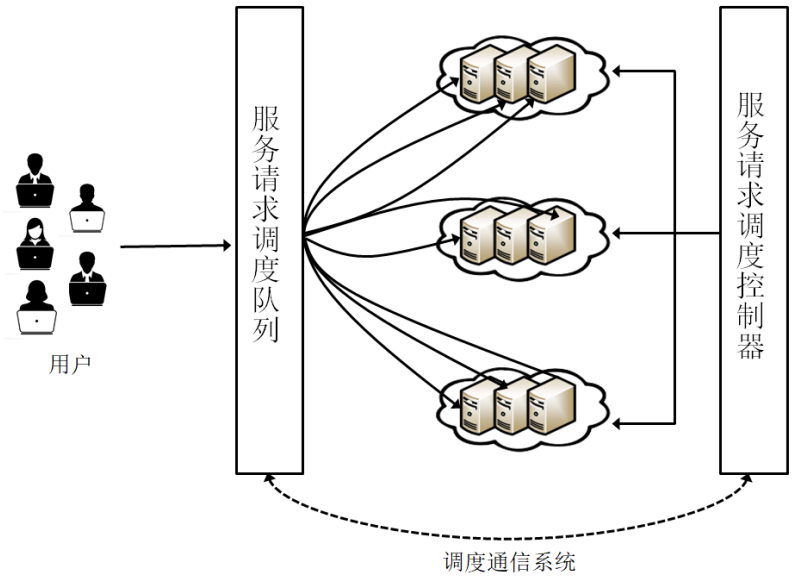



图 2.3 云服务平台

### 2.3.2 多服务器系统

多服务器系统模型的架构细节非常灵活，文献中对于组成多服务器系统的服务器的称呼也各种各样。例如在传统的服务器集群中，服务器被称为处理器[5][6][7]；而文献[39]将多服务器系统称为刀片中心(Blade Centers)，其中的服务器被称为服务器刀片(Blade Servers)；在文献[32]中，服务器被称为核心。为了统一起见，本文将这些刀片/处理器/核心都称为服务器。

为了让本文的研究能够适应于多种不同的情况，本节将多服务器系统建模为由单核服务器、同构多核服务器以及异构多核服务器组成的多样性系统。下面分别对单核服务器、同构多核服务器以及异构多核服务器进行介绍。

- **单核服务器**即该服务器有且只有一个处理器核心。该处理器核心有两种运行模式：睡眠模式和活动模式。当处理器核心执行服务请求时，处于活动模式；当处理器核心没有执行服务请求时，处于睡眠模式。则可以用$(f, v)$表示该处理器核心运行时的频率和电压，其中$f$表示该处理器核心的频率，$v$表示对应的电压。
- **同构多核服务器**指的是该服务器有多个相同的处理器核心。与单核服务器类似，该服务器也有两种运行模型，当服务器执行服务请求时处于活动模

式，当处理器核心没有执行服务请求时处于睡眠模式。假设该服务器有 $K$ 个同构处理器核心，则可以用$\{\sigma_1, \sigma_2, \dots, \sigma_K\}$表示该服务器的多个核心。其中每个核心有相同的频率/电压对，即$\sigma_1 = \sigma_2 = \cdots = \sigma_K = (f_1, v_1) = (f_2, v_2) = \cdots = (f_K, v_K)$，其中$f_k$表示处理器核心$\sigma_k$的频率，$v_k$表示对应的电压。

- **异构多核服务器**指的是该服务器有多个结构不同的处理器核心。若该服务器有 $K$ 个处理器核心，则该服务器可表示为$\{\theta_1, \theta_2, \dots, \theta_K\}$。由于该服务器的每个核心都不同构，本节用$(f_k, v_k)$表示处理器核心$\theta_k$的频率和电压，且满足$(f_1, v_1)! = (f_2, v_2)! = \cdots ! = (f_K, v_K)$，即每个处理器核心的频率/电压对都不完全相同。

### 2.3.3 收入模型

云提供商向用户提供云服务，并根据与用户签订的服务等级协议 SLA 和用户服务请求的完成情况收费。本文将云提供商的收入定义为所有用户服务请求收费的总和。由于用户的个性，用户的服务请求的属性也各不相同，这导致每个用户的服务价格和 SLA 都不同，因此不能以服务请求的单位收费，而需要以用户的单位对各个用户进行收费，然后将所有用户的收费累加作为云提供商的收入。则本文将云提供商的收入定义为：

$$Revenue = \sum_{n=1}^{N} \sum_{m=1}^{M_n} R(\tau_{n,m}, ft(\tau_{n,m})) \tag{2.4}$$

其中$Revenue$表示云提供商的收入，$N$为用户的数量，$M_n$为用户$n$的服务请求数量，$\sum_{m=1}^{M_n} R(\tau_{n,m}, ft(\tau_{n,m}))$为用户 $n$ 所有服务请求的收费，$R(\tau_{n,m}, ft(\tau_{n,m}))$为用户$n$第 $m$ 个服务请求的收费，$\tau_{n,m}$为用户$n$第 $m$ 个服务请求，$ft(\tau_{n,m})$为用户$n$第 $m$ 个服务请求的完成时间。

### 2.3.4 成本模型

云提供商的成本主要包括两个部分：向基础设施层租赁服务器的租赁费用和维持服务器运行的电费。其中，租赁费用主要由租赁的服务器数量以及租赁时间决定，而电费主要由服务器运行导致的功耗决定。接下来，本节分别对租赁费用和电费成本进行建模。

**租赁费用：**云提供商从基础设施服务提供商处租赁服务器，然后根据自己的服务类型构建服务环境以向用户提供云服务。由于服务器的租赁价格与服务器配置相关，而服务器根据带宽、处理器和显卡等软硬件资源的不同有多种配置，因此本节根据服务器配置不同对服务器的租赁费用进行单独计算。假设服务器的数量为 $N$，第 $n$ 个服务器的租赁价格为$r_n$，租赁时间为$d_n$，则云提供商的租赁成本为：

$$Rent = \sum_{n=1}^{N} (r_n * d_n) \tag{2.5}$$

**电费成本：**云提供商的电费成本由服务器运行产生的功耗决定，因此本节先定义服务器运行的功耗模型，再根据服务器功耗对电费成本进行计算。

服务器运行导致的功耗由两部分组成：动态功耗和静态功耗[40]，即：

$$p_{exe} = p_{sta} + p_{dyn} \tag{2.6}$$

其中$p_{exe}$为服务器运行导致的总功耗，$p_{sta}$表示静态功耗，即电流泄漏导致的功耗，$p_{dyn}$表示动态功耗，主要取决于服务器的运行时电压和频率。

由于动态功耗$p_{dyn}$在总功耗里占据主导地位，且静态功耗$p_{sta}$和服务器的活动无关，本文将静态功耗$p_{sta}$作为一个和服务器硬件相关的常数，主要对动态功耗$p_{dyn}$进行建模。当服务器$S_n$以频率/电压对$(f_n, v_n)$运行时，该服务器的动态功耗定义为：

$$p_{dyn} = \mu_n * C_n * v_n^2 * f_n \tag{2.7}$$

其中，$\mu_n$为服务器$S_n$的动态活动因子，$C_n$为服务器$S_n$的载入电容，$f_n$和$v_n$分别是服务器$S_n$运行时频率和电压[41]。

一般情况下，服务器运行电压和频率的关系为$v \propto f^a$，其中$a$为常量且大于0[42]。为了进一步的简化动态功耗的公式以方便计算，本文假设$v = bf^a$，其中$b$为常量因子，则动态能耗的公式可简化为：

$$p_{dyn} = \varphi * f_n^{\gamma} \tag{2.8}$$

其中，$\varphi = \mu_n * C_n * b^2$，$\gamma = 2 * a + 1$，具体常数的设置和服务器使用的处理器有关，例如，当设定$b = 1.16$，$\mu_n * C_n = 7.0$，$a = 0.5$，$\gamma = 2 * a + 1 = 2.0$，$\varphi = \mu_n * C_n * b^2 = 9.4192$时，此时通过公式（2.8）计算出的动态能耗与 Intel Pemtium M 处理器的实际功耗极其接近[43]。假设服务器$S_n$上有 $M$ 个服务请求，且$W_m$为服务请求$\tau_m$的指令条数，则服务器$S_n$上的能量消耗为：

$$E_n = \sum_{m=1}^{M} \left( \varphi * f_n^{\gamma-1} * W_m + p_{n,sta} * \frac{W_m}{f_n} \right) \tag{2.9}$$

其中，$p_{n,sta}$表示服务器$S_n$的静态能耗。若多服务器系统有 $N$ 个服务器，则云提供商的总能量消耗为：

$$E_{tol} = \sum_{n=1}^{N} E_n \tag{2.10}$$

计算出云提供商所有服务器消耗的能量后，本文使用实时电价模型来计算云提供商的电费成本。实时电价模型指的是不同时间段的电价不同，例如用电高峰期和低谷期的电价就明显不同。本文用$r(x)$表示实时电价模型的收费函数，$x$表示云提供商的总能耗，则电费成本可表示为：

$$Electricity = r(E_{tol}) \tag{2.11}$$

综合上述对云提供商租赁成本和电费成本的建模，本文将云提供商的成本定义为：

$$Cost = Electricity + Rent \tag{2.12}$$

其中，$Electricity$为云提供商的电费成本，$Rent$为租赁成本，这两者构成了云提供商的成本模型。

## 2.4 问题定义

基于上述云提供商的收入模型和成本模型，本文将云提供商的利润定义为收入减去成本，即：

$$Profit = Revenue - Cost \tag{2.13}$$

其中，$Profit$为云提供商的利润，$Revenue$为云提供商的收入，$Cost$为云提供商的成本。本文致力于保证用户满意度的前提下优化云提供商的利润，因此本文的研究问题可定义为：

$$Maximize: \ Profit$$

$$Subject \ \ to: \mathcal{S} \geq C$$

其中，云提供商利润$Profit$和用户满意度$\mathcal{S}$分别在公式（2.13）和（2.3）给出，$C$是一个常数，代表要保证的用户满意度。

## 2.5 本章小结

本章主要介绍了后续研究所需要的系统架构和模型，主要包括了三层云计算架构、用户模型及云提供商模型。首先介绍了由基础设施服务提供商、云服务提供商和用户组成的三层云计算架构，其中基础设施服务提供商提供硬件设施，云服务提供商租赁硬件设施并提供服务，用户向云服务提供商提交服务请求并付费；然后介绍了用户模型，主要包括了服务请求模型和用户满意度模型；最后介绍了云提供商模型，其中先是介绍了云服务平台模型和多服务器系统模型，然后介绍了收入和成本模型；最后基于这些模型给出了用户满意度约束下的云提供商利润优化问题的问题定义。

# 第三章 基于统计模型的用户满意度预测研究

众所周知，从商品的角度来看，商品质量和商品价格是影响用户满意度的两个重要因素，而云服务作为特殊的商品，云用户作为购买者自然也受服务价格和服务质量的影响。就像传统的计算机系统性能和能量平衡一样，低廉的服务价格和高服务质量很难同时得到满足：更高的服务质量通常需要更高的服务价格，而减少服务价格将不可避免地牺牲服务质量。显然，云提供商应该适当地平衡服务价格和服务质量以提高用户满意度。由于不同个性的用户对服务价格和服务质量的要求高度多样化，因此固定的平衡方案难以满足所有用户。

本节首先介绍了大五人格以及其测量方法，然后观察到用户的个性很大程度上影响了他/她在服务价格和服务质量之间的偏好。接着，本节基于问卷调查模拟了用户个性对他/她在服务价格和服务质量之间的偏好的影响，以及对指定服务的满意度。最后，本节根据用户的满意度模型来为每个用户定制服务请求属性以捕捉用户对服务价格和服务质量的偏好。接下来的章节里，本文将通过调度为每个用户提供定制的平衡方案以提高用户服务体验。

## 3.1 大五人格与测量方法

个性指的是思维、情感和行为的特征模式中的个体差异[44]，这种差异性致使不同个性的人在面对相同情况时可能有不同的反应，对人们的选择有很大的影响。它通常用表 3.1 中总结的大五人格[45]来描述，大五人格是人格心理学中最具有影响力的人格模型之一，被众多国内外心理学家所认可。大五人格中包含的五项人格特质普遍适用于各类人群和种族，不会因为地域、文化和语言的不同而有所改变。大五人格包含的这五个因素分别为外倾性（E）、宜人性（A）、责任心（C）、神经质或情绪稳定性（N）和开放性（O），表 3.1 列举了这五种人格的特质及其描述，下面对这五种人格进行简单介绍。

- 外倾性（Extroversion）表示人际互动的数量和密度、对刺激的需要以及获得愉悦的能力。高外倾性主要表现为擅长交际、热心助人、乐群和乐于表

达等，低外倾性则表现为沉默寡言、自省和独断等。

- 宜人性（Agreeableness）描述了个体对周围人和事物赞成或敌对的程度。高宜人性表现为合作、利他、顺从和同理心等特点，而低宜人性则表现为自私、冷漠、多疑和对抗等。
- 责任心（Conscientiousness）描述了个体抑制冲动和约束行为的能力。高责任心表现为谨慎小心、尽职尽责、井井有条和三思而行等，低责任心则表现为懒散懈怠、考虑不周、冲动行事和缺乏规划等。
- 神经质或情绪稳定性（Neuroticism）描述了个体情绪控制和承受压力的能力。高神经质表现为焦虑、易怒、暴躁和紧张等，而低神经质则表现为冷静、淡定、平和及稳定等。
- 开放性（Openness）描述了个体的认知风格，高开放性个体表现为思维抽象、好奇心强、喜欢追求新颖和不受常规限制等，而低开放性个体表现为追求实际、偏爱常规、比较保守和传统等。

表 3.1 大五人格特质及其描述

| 特质 | 描述 |
| --- | --- |
| 外倾性（E） | 表现出热情、健谈、果断、活跃、冒险和乐观等特点 |
| 宜人性（A） | 具有信任、利他、直率、依从、谦虚和移情等特点 |
| 责任心（C） | 具有认真、公正、条理、尽职、勤奋、自律、谨慎和克制等特点 |
| 神经质或情绪稳定性（N） | 具有平衡焦虑、敌对、压抑、自我意识、冲动和脆弱等情绪的特质，即具有保持情绪稳定的能力 |
| 开放性（O） | 具有想象、审美、情感丰富、求异、创造和富有好奇心等特质 |

为了评估人格，通常使用一个带有十个问题的问卷，称为十项人格量表（Ten Item Personality Inventory，TIPI）[46]。通过使用 TIPI 问卷调查，用户的个性可以通过五大特征得分（每个范围从 1 到 7）的向量来描述，本节将该向量命名为个性评分。TIPI 问卷的具体内容见表 3.2。

表 3.2 TIPI 问卷

| 非常不同意 1 分 | 比较不同意 2 分 | 有点不同意 3 分 | 中立 4 分 | 有点同意 5 分 | 比较同意 6 分 | 非常同意 7 分 |
|---|---|---|---|---|---|---|
| 你认为自己的个性是为： | | | | 分数 | | |
| 外向的，精力充沛的 | | | | $\mathrm{E}_p$ | | |
| 招人喜欢的，友善的 | | | | $\mathrm{A}_p$ | | |
| 可信赖的，自律的 | | | | $\mathrm{C}_p$ | | |
| 冷静的，情绪稳定的 | | | | $\mathrm{N}_p$ | | |
| 易接受新事物的，常有新想法的 | | | | $O_p$ | | |
| 内向的，安静的 | | | | $\mathrm{E}_n$ | | |
| 爱批评人的，爱争吵的 | | | | $A_n$ | | |
| 条理性差的，粗心的 | | | | $\mathrm{C}_n$ | | |
| 忧虑的，易心烦的 | | | | $N_n$ | | |
| 遵循常规的，不爱创新的 | | | | $O_n$ | | |

如表 3.2 所示，问卷调查的参与者需要根据自己的性格和认可程度填写对应的分数，其中$E_p$、$A_p$、$C_p$、$N_p$和$O_p$为大五人格的正向量，对应的$E_n$、$A_n$、$C_n$、$N_n$和$O_n$为大五人格的负向量，则每个参与者的大五人格评分为$\{E,A,C,N,O\}=\left\{\frac{\mathrm{E}_p+(8-\mathrm{E}_n)}{2},\frac{A_p+(8-A_n)}{2},\frac{C_p+(8-C_n)}{2},\frac{N_p+(8-N_n)}{2},\frac{O_p+(8-O_n)}{2}\right\}$。

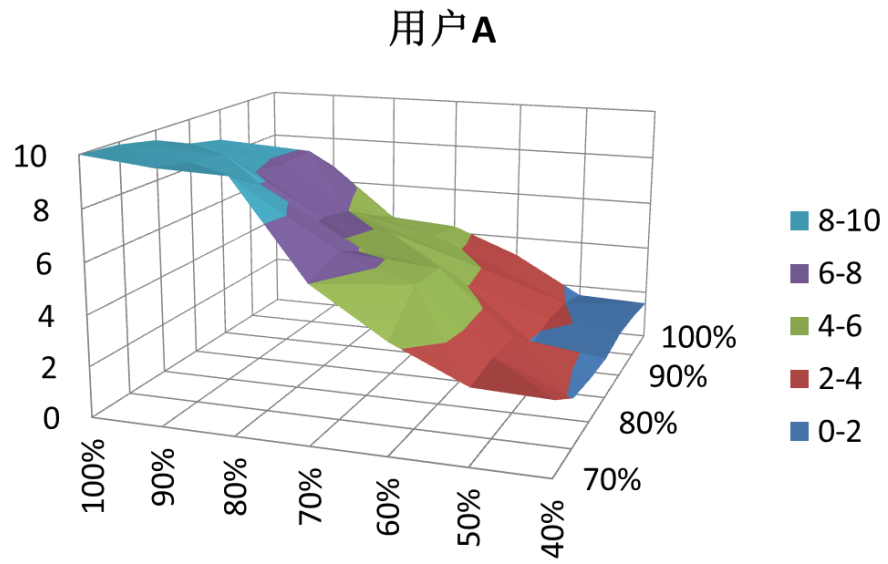
## 3.2 人格对于用户满意度的影响探究

得到了用户的大五人格评分后，本节将进一步探讨人格对于用户满意度的影响。该研究的基本原理是通过考虑服务质量及其相应的服务价格来向每个用户询问不同服务质量-服务价格水平的满意度评级。然后，根据用户对不同服务质量-服务价格水平的满意度分数，得到该用户的满意度模型。

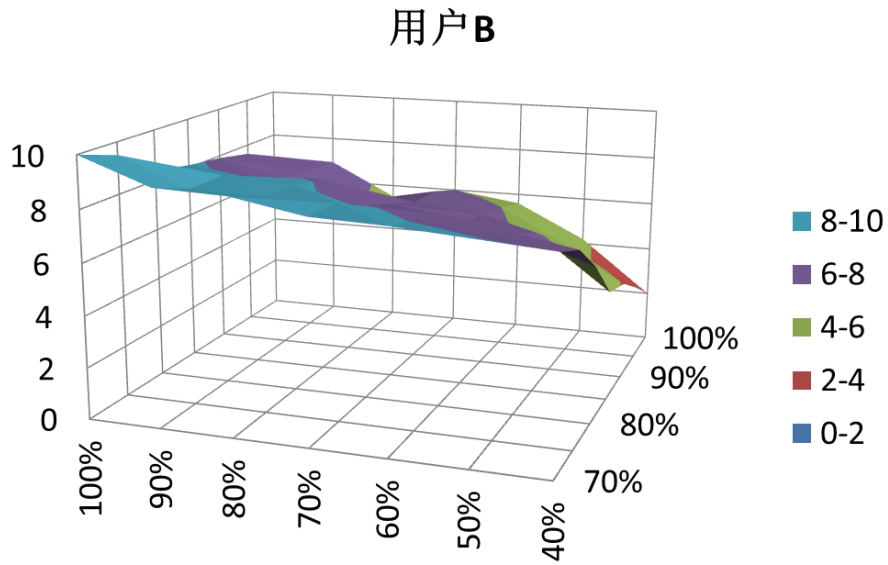
首先，为了获得一组服务质量级别，本节改变了服务期间的系统响应时间，即用户服务请求的响应时间。本节将基线响应时间设置为服务器可以支持的最佳响应，然后基于基线响应时间设置六个级别的响应时间降级，即-0%、-10%、-20%、-30%、-40%、-50%和-60%。对于服务价格，本节将原始服务价格设置为基线，然后相对应的设置六个级别的价格降级，即-0%、-5%、-10%、-15%、-20%、-25%和-30%。结合六个级别的服务质量和六个级别的服务价格，可以得到一共 36 个级别的服务质量-服务价格水平。然后，针对每个服务质量-服务价格水平级别分别收集用户的满意度分数，范围从 1（最低满意度）到 10（最高满意度）。得分

最高的级别称为最佳服务质量-服务价格水平。

图 3.1 展示了两个用户对于不同服务质量-服务价格水平所得到的满意度分数，x 坐标是服务价格，y 坐标是服务质量，z 坐标是用户的服务满意度。用户 A 具有较强的亲和性，而用户 B 具有较强的外向性。如图所示，用户 A 的满意度与服务质量呈正相关。对于用户 B 来说，服务价格对满意度有很大的影响，而服务质量即响应时间则不那么重要。由图 3.1 可以看出，用户的个性对他们的满意度得分有很大的影响。




(a) 用户 A 的满意度模型




(b) 用户 B 的满意度模型

图 3.1 用户服务满意度分数

## 3.3 以人格为导向的用户满意度建模

鉴于观察到人格对用户偏好有很大影响，本节建立统计模型来推断用户满意度得分。如图 3.2 所示，该模型将服务价格$price$、服务质量$qos$和该用户的大五人格分数$score$作为输入，输出该用户对于服务价格和服务质量的偏好$p$以及该用户的满意度函数$\mathcal{S}$。

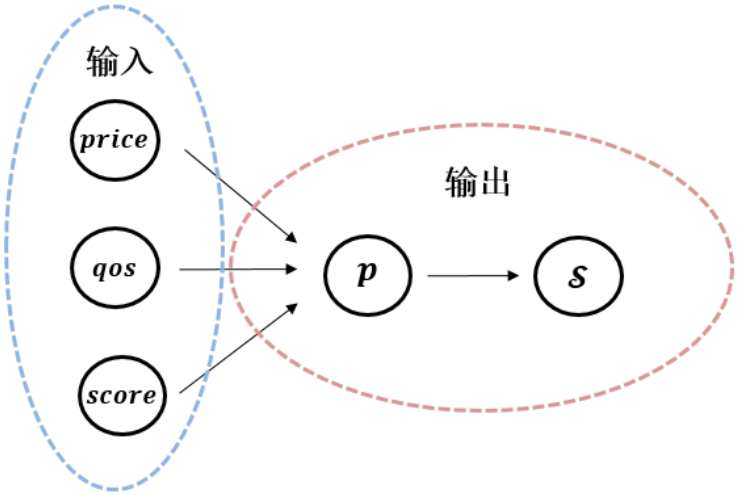



图 3.2 用户服务满意度预测模型

本文在用户模型中对用户偏好和用户满意度进行了定义，并通过公式（2.2）（2.3）分别对用户偏好和用户满意度进行了建模。接下来本节将根据问卷调查结果对公式（2.3）中的$\alpha_1$和$\alpha_2$进行估算，以得到用户偏好和用户满意度的关系。本节将参与者的数量表示为$N_e$，将服务质量-服务价格水平的数量表示为$N_l$。对于参与者$i$（$i \in [1, N_e]$），本节将他/她的偏好权重表示为$\gamma_i = [\gamma_{i,1}, \gamma_{i,2}]$，对于每个服务质量-服务价格水平$j$（$j \in [1, N_l]$），该参与者$i$的服务价格、服务质量、偏好和满意度得分分别表示为$price_{i,j}$、$qos_{i,j}$、$p_{i,j}$和$\mathcal{S}_{i,j}$。则参数$\alpha_1$和$\alpha_2$可以通过解决以下的均方误差来获得：

$$\min_{\alpha_1,\alpha_2,\gamma_1,\ldots,\gamma_{N_e}} \sum_{i=1}^{N_e}\sum_{j=1}^{N_l}(\mathcal{S}_{i,j} - \alpha_1 - \alpha_2 * p_{i,j})^2 \tag{3.1}$$

其中：

$$p_{i,j} = \gamma_i * qos_{i,j} + (1 - |\gamma_i|_1) * price_{i,j} \tag{3.2}$$

由于公式（3.1）有过多的未知变量，无法直接解决。因此，本节开发了一种迭代算法来解决它。本节首先为每位参与者的偏好权重设置一个初始值，然后每个参与者的偏好$p_{i,j}$可以由公式（3.2）计算出，这样公式（3.1）就变成了标准的线性回归，可以通过求解以下线性回归来获得$\alpha_1$和$\alpha_2$的值：

$$\min_{\alpha_1,\alpha_2} \sum_{i=1}^{N_e} \sum_{j=1}^{N_l} (\mathcal{S}_{i,j} - \alpha_1 - \alpha_2 * p_{i,j})^2 \tag{3.3}$$

通过公式（3.3）可以获取$\alpha_1$和$\alpha_2$的值，然后给定$\alpha_1$和$\alpha_2$，对于参与者$i$公式(3.1)就变成了：

$$\min_{\gamma_i} \sum_{j=1}^{N_l} (\frac{\mathcal{S}_{i,j} - \alpha_1}{\alpha_2} - (1 - |\gamma_i|_1) * price_{i,j} - \gamma_i * qos_{i,j})^2 \tag{3.4}$$

同样，公式（3.4）可以通过线性回归求解器解决，然后我们可以获取参与者$i$更新后的偏好权重$\gamma_i'$。如果更新后的$\gamma_i'$和初始值$\gamma_i$足够接近，则更新$\alpha_1$和$\alpha_2$的值并对下一个参与者进行迭代算法。若所有参与者$i$（$i \in [1, N_e]$），$\gamma_i'$和$\gamma_i$都足够接近，则算法停止且我们获得了所有人的偏好权重以及参数$\alpha_1$和$\alpha_2$的值。迭代算法的具体步骤和伪代码如算法 3.1 所示。

**算法 3.1 迭代算法**

输入：$N_e$个参与者对$N_l$个服务质量-服务价格水平的满意度得分，$\{(price_{1,1}, qos_{1,1}, \mathcal{S}_{1,1}), \ldots, (price_{N_e,N_l}, qos_{N_e,N_l}, \mathcal{S}_{N_e,N_l})$;

输出：$N_e$个参与者的偏好权重$\gamma_i$，参数$\alpha_1$和$\alpha_2$；

/* 初始化$N_e$个参与者的偏好权重$\gamma$*/

1: **for** *i = 1* **to** $N_e$ **do**

/* 将第 *i* 个参与者的偏好权重$\gamma_i$置为[0.25,0.25]*/

2: $\gamma_i = [0.25,0.25]$;

3: **end for**

/* 迭代求解所有参与者的偏好和参数$\alpha_1$和$\alpha_2$*/

4: **do**{

/* 设置迭代算法结束标志，一次迭代结束后仍为 1 则结束迭代算法 */

5: *flag = 1;*

/* 根据目前迭代中的参与者偏好权重计算参数$\alpha_1$和$\alpha_2$的值*/

6: 将$\gamma_1, \ldots, \gamma_{N_e}$带入公式（3.1）得到公式（3.3）；

7: 使用线性回归求解器解公式（3.3）得到参数$\alpha_1$和$\alpha_2$的值；

/* 根据目前迭代中的参数$\alpha_1$和$\alpha_2$的值更新所有参与者的偏好权重 */

8: **for** *i = 1* **to** $N_e$ **do**

/* 根据目前迭代中的参数$\alpha_1$和$\alpha_2$的值计算参与者 *i* 的偏好权重 */

9: 将参数$\alpha_1$和$\alpha_2$的值带入参与者 *i* 的公式（3.1）得到公式（3.4）；

10: 通过线性回归求解器解公式（3.4）得到$\gamma_i'$;

/* 如果$\gamma_i'$和$\gamma_i$不够接近则应该继续迭代 */

11: **if** $|\gamma_i - \gamma_i'| > k$ **then**

/* 将迭代标志置为 0，表示应该继续迭代 */

12: *flag = 0;*

13: **end if**

/* 更新参与者 *i* 的偏好权重 */

14: $\gamma_i = \gamma_i'$;

15: **end for**

/* 若不满足所有参与者的$\gamma_i$足够接近$\gamma_i'$，则继续迭代 */

16: }**while**(*flag!=1*)

17: **return** $\alpha_1, \alpha_2, \gamma_1, \ldots, \gamma_{N_e}$;

算法 3.1 的输入为问卷调查中$N_e$个参与者对$N_l$个服务质量-服务价格水平的满意度得分以及对应的服务质量和服务价格，用$(price_{i,j}, qos_{i,j}, \mathcal{S}_{i,j})$表示参与者$i$对服务质量-服务价格水平$j$的满意度以及该水平对应的服务质量和服务价格。输出为$N_e$个参与者的偏好权重$\gamma_i$、参数$\alpha_1$和$\alpha_2$的值。该算法首先进行初始化工作，

将所有参与者的偏好权重设置为$[\frac{1}{4},\frac{1}{4}]$（第 1-3 行）。初始化工作完成之后，该算法开始迭代过程（第 4-16 行）。首先，该算法设置了一个迭代标志用来判断迭代算法是否应该结束，若迭代后迭代标志仍为 1，则迭代结束，否则进入下一次迭代（第 5 行）。然后，该算法利用参与者的偏好权重和公式（3.3）计算出参数$\alpha_1$和$\alpha_2$的值（第 6-7 行），再利用参数$\alpha_1$和$\alpha_2$的值和公式（3.4）得到参与者更新后的偏好权重$\gamma_i'$（第 8-10 行），通过对比更新后的偏好权重$\gamma_i'$和原始偏好权重$\gamma_i$确定迭代状态（第 11-13 行），并更新参与者的偏好权重（第 14 行）。如果所有参与者更新后的偏好权重$\gamma_i'$和原始偏好权重$\gamma_i$都足够接近，那么迭代算法结束（第 15-16 行），并输出$N_e$个参与者的偏好权重$\gamma_i$、参数$\alpha_1$和$\alpha_2$的值（第 17 行）。

由算法 3.1 我们得到了每个参与者的偏好权重，接下来本节结合问卷调查得到的每个参与者的个性得分进一步建立统计模型来探索个性得分和偏好权重的关系。对于参与者$i$，本节用$g_i = [g_{i,E}, g_{i,A}, g_{i,C}, g_{i,ES}, g_{i,O}\ ]$来表示他/她的人格评分，其中五个参数分别代表五个人格特质 E、A、C、ES 和 O 的得分，则对于$N_e$个参与者，本节可以用$N_e \times 5$的矩阵$M$表示所有参与者的人格评分，用$N_e \times 1$的向量 $N$ 表示所有参与者的偏好权重。本节建立一个线性回归模型根据人格得分对用户偏好权重进行估计。参数 X（$5 \times 1$的向量）可以通过求解公式（3.5）来得到。

$$\min_{\mathrm{X}} \|N - M \times \mathrm{X}\|^2 \tag{3.5}$$

根据公式（3.5），用户$i$的偏好权重可以通过用户的人格评分$g_i$乘以参数 X 得到。至此，本节根据用户个性评分可以得到每个用户的满意度函数中的所有参数，基于该满意度函数，可以预测用户对于所有服务价格-服务质量层级的满意度分数。

## 3.4 基于用户满意度模型确定用户服务请求属性

在得到用户的偏好和满意度函数后，本节将根据用户的个性设定该用户服务请求的属性，以让用户得到满意的服务。由上文可知，用户服务请求的属性为

$\{W_m, p_m, D_m^{des}, \alpha_m, D_m\}$，其中服务请求的指令条数由服务请求本身确定，其他属性由用户个性确定。为了保证用户能够得到满意的服务，本节将该用户的用户满意度$\mathcal{S}$设定为 10，然后根据公式（2.2）（2.3）可以得到该用户最满意的服务价格$price$和服务质量$qos$，将这两个变量的值作为该用户服务请求的服务价格$p_m$和渴望完成时间$D_m^{des}$。为了得到该用户对于低服务质量的容忍度$\alpha_m$，即服务质量下降导致的用户满意度衰减率，可将服务价格$p_m$带入公式（2.2），再将公式（2.2）带入公式（2.3），得到：

$$\mathcal{S} = \alpha_1 + \alpha_2 * (\gamma * qos + (1 - |\gamma|_1) * p_m) \tag{3.6}$$

公式（3.6）可化简为：

$$\mathcal{S} = \alpha_2 * \gamma * qos + \ (\alpha_1 + \alpha_2 * (1 - |\gamma|_1) * p_m) \tag{3.7}$$

其中，$p_m$为已知变量，$\alpha_1$、$\alpha_2$、$\gamma$和$|\gamma|_1$可通过本节提出的用户满意度预测模型和该用户个性分数得到，则$(\alpha_1 + \alpha_2 * (1 - |\gamma|_1) * p_m)$和$\alpha_2 * \gamma$均为常量，因此该用户对于低服务质量的容忍度$\alpha_m$设定为$\alpha_2 * \gamma$。服务请求截至时间$D_m$为用户满意度变为 0 的时间，因此将$\mathcal{S} = 0$带入公式（3.6），可以得到：

$$D_m = \frac{(\alpha_1 + \alpha_2 * (1 - |\gamma|_1) * p_m)}{\alpha_2 * \gamma} \tag{3.8}$$

至此，本节通过该用户的个性分数得到了该用户服务请求的所有属性，这样该服务请求不但能够捕捉到不同用户对于服务价格-服务质量的偏好，还能够得到该用户对于低服务质量的容忍度，便于接下来的章节在用户满意度约束下进行云提供商的利润优化。

## 3.5 本章小结

本章考虑了不同用户对服务价格和服务质量的偏好的高度多样化，解决了个性用户满意度预测问题。首先，本章通过问卷调查深入探究了用户个性即大五人格评分对用户偏好的影响。根据问卷调查结果，本章建立了用户满意度预测模型

来估计用户对于指定服务质量-服务价格的满意度以及用户的偏好，并设计了迭代算法来获取预测模型的参数。最后，本节根据用户个性评分和满意度预测模型得到了能够捕捉该用户个性需求的服务请求属性，这将帮助后续章节为用户提出定制化的调度策略，以在保证用户满意度前提下优化云提供商利润。

# 第四章 基于整数线性规划的云提供商利润优化研究

上一章节介绍了如何通过统计模型得到不同用户对于服务质量和服务价格的偏好，这会导致用户服务请求的复杂多样性。基于这种用户服务请求的多样性，本章使用整数线性规划 ILP 对利润驱动的高用户满意度调度方法进行了研究，解决了用户满意度约束下的云提供商利润优化问题。

具体来说，本章首先将云提供商使用的混合服务器系统转换为虚拟单核服务器系统以减少调度复杂性；然后，使用 ILP 对所研究的最优化问题进行建模和求解，并提出了基于 ILP 的云提供商利润优化算法；最后，指出了基于 ILP 的利润优化算法的不足之处，提出了设计启发式利润优化算法的必要性。

## 4.1 多核服务器向虚拟单核服务器的转换

由上述章节可知，本文的云提供商使用的服务器系统为单核服务器、多核同构服务器和多核异构服务器组成的多服务器系统，这要求调度程序不但要决定服务请求分配在哪个服务器，而且还要决定服务请求分配在该服务器中的哪个处理器核心，最后还需要决定服务请求在该处理器核心的执行顺序，这三个层次的调度大大增加了调度程序的算法复杂度。因此，本节将多核服务器模型转换为虚拟单核服务器，从而减少调度程序的调度层数，降低算法复杂度。

假设云提供商的多服务器系统由$N$个单核服务器、$M$个多核同构服务器和$K$个多核异构服务器组成，则该多服务器系统可表示为：$\{S_1, S_2, \dots, S_{N+M+K}\}$。其中，$N$个单核服务器已经是单核服务器，不需要进行转换，本节主要将$M$个多核同构服务器和$K$个多核异构服务器转换成虚拟单核服务器。

对于多核同构服务器$S_m = \{\sigma_1, \sigma_2, \dots, \sigma_L\}$，其中$\sigma_l$为服务器$S_m$的第$l$个处理器核心，且每个处理器核心的频率/电压对都为$(f, v)$。本节做以下转换$S_m = \{\sigma_1, \sigma_2, \dots, \sigma_L\} = \{\hat{S}_1, \hat{S}_2, \dots, \hat{S}_L\}$，其中$\hat{S}_1 = \hat{S}_2 = \cdots = \hat{S}_L = (f, v)$，即$\hat{S}_l$为只有一个处理器核心的虚拟单核服务器，且处理器核心的性能和原多核同构服务器$S_m$的处理器核心相同。

对于多核异构服务器$S_k = \{\theta_1, \theta_2, \dots, \theta_J\}$，其中$\theta_j$为服务器$S_k$的第$j$个处理器核心，且$\theta_j = (f_j, v_j)$，即每个处理器核心都有不同的频率/电压对。本节对该多核异构服务器做以下转换$S_k = \{\theta_1, \theta_2, \dots, \theta_J\} = \{\hat{S}_1, \hat{S}_2, \dots, \hat{S}_J\}$，其中$\hat{S}_j = (f_j, v_j)$，即转换后的第$j$个虚拟单核服务器的频率/电压对与多核异构服务器$S_k$的第$j$个处理器核心相同。

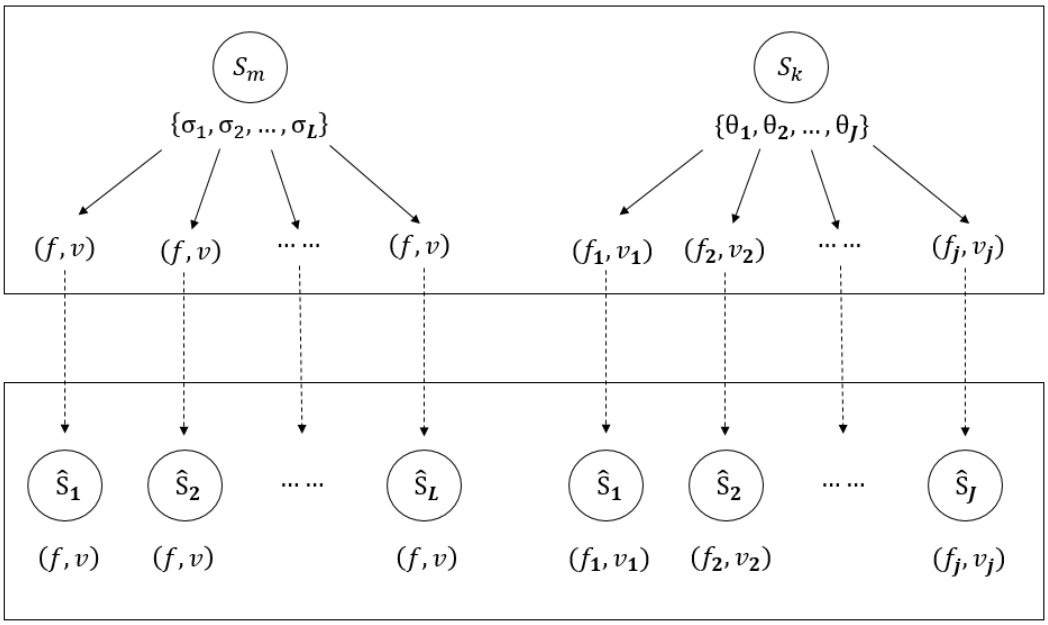



图 4.1 多核服务器向虚拟单核服务器的转换过程

图 4.1 描述了上述多核服务器向虚拟单核服务器的转换过程。则云提供商的多服务器系统$\{S_1, S_2, \dots, S_{N+M+K}\}$可转换为单核虚拟服务器系统$\{\hat{S}_1, \hat{S}_2, \dots, \hat{S}_{\mathcal{H}}\}$。其中$\mathcal{H}$为所有服务器的处理器核心的总数，可表示为：

$$\mathcal{H} = \sum_{i=1}^{N+M+K} S_{i,core} \tag{4.1}$$

其中，$N + M + K$表示所有服务器的数量，$S_{i,core}$表示服务器$S_i$的处理器核心的数量。

在经过多核服务器系统向虚拟单核服务器系统的转换后，原先三个层次的调度就转换为两个层次的调度，调度程序只需要决定服务请求在虚拟单核服务器的分配和执行顺序。若无特别说明，随后章节提到的服务器指的都是经过转换的虚拟单核服务器。

## 4.2 基于整数线性规划的利润优化研究

本节首先使用 ILP 对所研究的最优化问题进行建模和求解，然后提出了基于 ILP 的云提供商利润优化算法，随后指出基于 ILP 的利润优化算法的不足之处，并提出了设计启发式优化算法的必要性。

### 4.2.1 整数线性规划问题建模和求解

要想使用 ILP 求解最优化问题，首先需要将问题建模为 ILP 能解的一般形式，即给出求解问题的目标函数和相关约束的形式化表示，然后使用 ILP 求解器就可以求解所研究的最优化问题。本节首先给出问题描述，然后介绍目标函数和相关约束的形式化表示。

**问题描述：**给定一个由用户服务请求$\tau_m = \{W_m, p_m, D_m, \alpha_m, D_m\}$组成的集合$\mathbf{\Gamma}$和一组由服务器$S_n = \{v_n, f_n\}$组成的多服务器系统，其中$1 \le m \le M$，$1 \le n \le N$，$v_n$和$f_n$分别为服务器$S_n$的电压和频率。目标是找到服务请求应在的服务器位置以及所有服务请求$\tau_m$（$m \in [1, M]$）的开始执行时间，使得云提供商的利润最大化。在给出 ILP 问题的目标函数和相关约束的形式化表示之前，本节先引入形式化研究问题和相关约束时需要用到的两个变量：

$$A_{n,m} = \begin{cases} 1 & \text{如果服务请求}\tau_m\text{被分配到服务器}S_n \\ 0 & \text{如果服务请求}\tau_m\text{被分配到其他服务器} \end{cases} \tag{4.2}$$

$$O_{i,j} = \begin{cases} 1 & \text{如果服务请求}\tau_i\text{在服务请求}\tau_j\text{之后执行} \\ 0 & \text{如果服务请求}\tau_i\text{在服务请求}\tau_j\text{之前执行} \end{cases} \tag{4.3}$$

其中$1 \le n \le N$，$1 \le m \le M$，$1 \le i, j \le M$，$N$为服务器的个数，$M$ 为用户服务请求的数量。

**目标函数：**本文旨在保证用户满意度的前提下，优化云提供商的利润，因此本节的目标函数自然是最大化利润。根据本文在第二章对云提供商的建模和问题定义，ILP 的目标函数可表示为：

$$Maximize: Profit$$

**相关约束：**相关的约束条件为：

1.每个服务请求都只能被且仅被分配到一个服务器，即：

$$\sum_{n=1}^{N} A_{n,m} = 1 \tag{4.4}$$

2.云提供商接受的服务请求都能够盈利，即：

$$b_m + \sum_{n=1}^{N} A_{n,m} * \frac{W_m}{f_n} \leq D_m \tag{4.5}$$

其中$b_m$是服务请求$\tau_m$的开始执行时间，$W_m$是服务请求$\tau_m$的指令条数，$f_n$是服务器$S_n$的执行频率，$D_m$是服务请求的截至日期。

3.云提供商必须保证用户满意度，令R为要保证的用户满意度，则：

$$\frac{\sum_{m=1}^{M} S_m}{M} \geq \mathrm{R} \tag{4.6}$$

其中$S_m$为用户 $m$ 的满意度，可通过公式（2.3）得到。

4.服务器同一时刻只能执行一个服务请求，则：

$$bmax_{(i,j)} = O_{j,i} * b_i + O_{i,j} * b_j \tag{4.7}$$

$$h_{i,j} = H * \left(b_i - bmax_{(i,j)}\right) + b_j \tag{4.8}$$

$$b_i - h_{i,j} \geq \sum_{n=1}^{N} A_{n,j} * \frac{W_j}{f_n} \tag{4.9}$$

$$g_{i,j} = H * \left(b_j - bmax_{(i,j)}\right) + b_i \tag{4.10}$$

$$b_j - g_{i,j} \geq \sum_{n=1}^{N} A_{n,i} * \frac{W_i}{f_n} \tag{4.11}$$

其中$b_i$和$b_j$分别是服务请求$\tau_i$和服务请求$\tau_j$的开始执行时间，$bmax_{(i,j)}$是$b_i$和$b_j$中的较大值，$H$是一个足够大的常数，$h_{i,j}$和$g_{i,j}$为两个辅助变量。

### 4.2.2 基于整数线性规划的利润优化算法

所提出的利润优化算法对于用户服务价格的调整基于基准价格，而不同文献中基准价格的定价策略可能不同，因此为了让算法能够适应于不同的定价策略，本节首先将调度向量 **T** 定义为：

$$\mathbf{T} \triangleq [T_1, T_2, \dots, T_K] \tag{4.12}$$

其中 $K$ 是调度范围，$T_k$是调度范围 $K$ 的第 $k$ 个价格周期。为了适应不同定价策略导致的基准价格的变化，每个价格周期$T_k$的基准价格都可能不同，价格周期$T_k$的大小取决于云提供商使用的定价策略。对于静态定价，价格的波动不频繁且波动幅度较小，则价格周期$T_k$较大，可能是一个月、半年乃至一年。对于基准价格波动较大且频繁的动态定价，价格周期$T_k$可以是几天、几小时甚至是几秒。

对于调度范围中的每个价格周期，本节首先使用第三章提出的用户满意度预测模型预测出用户服务请求的各项属性；然后动态更新要分配的用户请求集 **Γ**、服务器状态和已分配请求的完成状态；最后采用上节描述的 ILP 技术来解决云提供商利润优化问题。由于用户会随机且动态的到来，上述步骤每隔一段时间都会执行一次，执行间隔时间取决于云提供商要求的调度精度和调度程序的计算能力。该方案的伪代码如算法 4.1 所示。

算法 4.1 的输入是用户服务请求集合 **Γ**、服务器集合$\{S_1, S_2, \dots, S_n\}$和调度向量 $\mathbf{T} \triangleq [T_1, T_2, \dots, T_K]$，输出是云提供商的总利润、用户平均满意度和每个调度时隙的调度结果。该算法首先根据云提供商的定价策略将整个调度分为 $K$ 个价格周期，在每个价格周期更新基准价格并划分调度时隙（第 1-3 行）。然后，该算法对新到来的用户进行满意度预测，得到让用户满意的服务请求参数，并将这些用户的服务请求放入服务请求集合 **Γ** 中等待调度（第 4-6 行）。在调度之前，该算法对当前系统的运行状态进行检测，包括服务器的运行状态和服务器上服务请求的执行状态（第 7-8 行）。在第 9-10 行，该算法使用 ILP 求解器获得服务请求集合到服务器的调度表，并根据公式（2.13）计算此次调度产生的

利润。当调度结束后，该算法首先根据调度结果和公式（2.3）计算每个用户的满意度并累加到总用户满意度（第 11-15 行），然后将利润累加到总利润中并输出该调度表（第 16-19 行）。最后，该算法输出总利润和用户平均满意度（第 20-21 行）。

**算法 4.1 基于整数线性规划的利润优化算法**

输入: 用户服务请求集合 $\boldsymbol{\Gamma}$，服务器集合$\{S_1, S_2, \dots, S_n\}$，调度向量$\mathbf{T} \triangleq [T_1, T_2, \dots, T_K]$;

输出: 云提供商总利润，用户平均满意度，每个调度时隙的调度结果;

```
/* 分别在云提供商的每个价格周期进行调度 */
for i = 1 to K do
    根据云提供商的定价策略更新基准价格
    /* 将每个价格周期分为 P 个调度时隙 */
    for j = 1 to P do
        使用公式（3.5）计算新到来用户的偏好;
        根据用户偏好设置新到来用户的服务请求属性;
        更新用户服务请求集合 Γ ;
        更新服务器状态;
        更新服务请求执行状态;
        /* 产生调度表并计算当前调度产生的利润 */
        通过 ILP 求解器产生从服务请求集合到服务器集合的调度表;
        使用公式（2.13）计算当前调度产生的利润为 profit;
        num=Size(Γ);
        for k=1 to num do
            使用公式(2.3)计算当前调度用户 k 的满意度 satisfaction;
            TSatisfaction = TSatisfaction + satisfaction;
        end for
        TProfit = TProfit + profit;
        TNum = TNum+num;
        输出调度表;
    end for
end for
return TProfit, TSatisfaction/ TNum;
```

## 4.3 基于整数线性规划的利润优化方案的不足

研究表明，解决任意的 ILP 都是 NP-hard 问题[47]。当服务请求和服务器个数较少时，ILP 能够有效的得到最优解，也就是产生最大利润的调度。但是随

着服务请求和服务器的数量逐渐增大，基于 ILP 的利润优化算法会导致两个问题。第一个问题是，当存在大量服务请求和服务器时，ILP 会消耗过多的计算资源，这会增加云提供商的成本。此外，当价格周期很小时，在下一个价格周期之前可能无法解决 ILP 利润优化问题，这不适合动态定价机制的应用。因此，本文有必要设计一个时间复杂度更低的启发式算法。

## 4.4 本章小结

本章主要在个性用户服务请求的多样性情况下，解决了用户满意度约束下的云提供商利润优化问题。首先，本章通过将复杂服务器系统转换为虚拟单核服务器系统以降低调度复杂度；然后对优化问题进行 ILP 的建模和求解，并设计了基于 ILP 的云提供商利润优化算法；最后，指出了该算法的不足之处，提出了设计启发式利润优化算法的必要性。

# 第五章 基于价值评估和交叉熵的云提供商利润优化研究

由上文可知，基于 ILP 的利润优化方案虽然能够实现用户满意度约束下的云提供商利润最大化，但过高的时间复杂度给云提供商带来了额外的负担，也无法适用于定价周期很短的云提供商。为了解决这两个问题，本章首先探讨了服务请求的分配以及执行顺序对云提供商利润和用户满意度的影响，为启发式利润优化算法的设计提供理论支持；然后，介绍了现值、机会成本和交叉熵的相关概念以及在本文中的应用，并基于它们设计了一个启发式算法以降低计算复杂度并加快调度速度。

## 5.1 启发式利润优化算法的理论支持

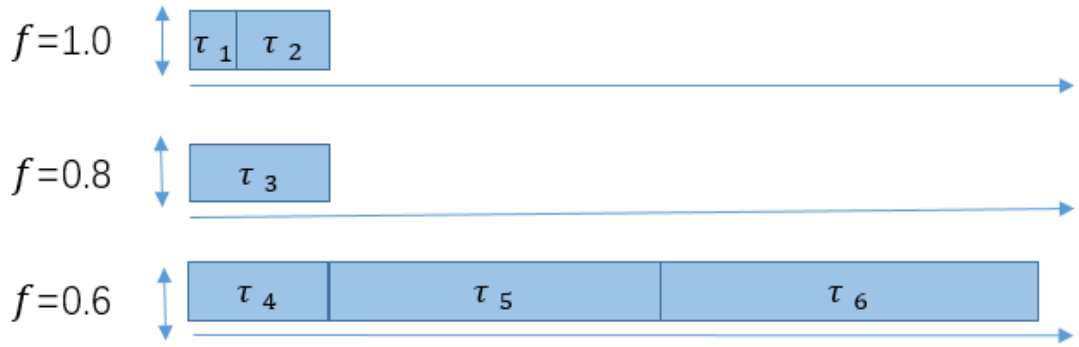
本节探索了服务请求的调度对云提供商利润以及用户满意度的影响，为随后提出的启发式利润优化算法提供理论支持。服务请求的调度主要包括两部分：服务请求在多服务器集群上的分配以及服务请求在各个服务器上的执行顺序，本节分别对这两部分进行讨论。

### 5.1.1 服务请求分配对云提供商和用户的影响

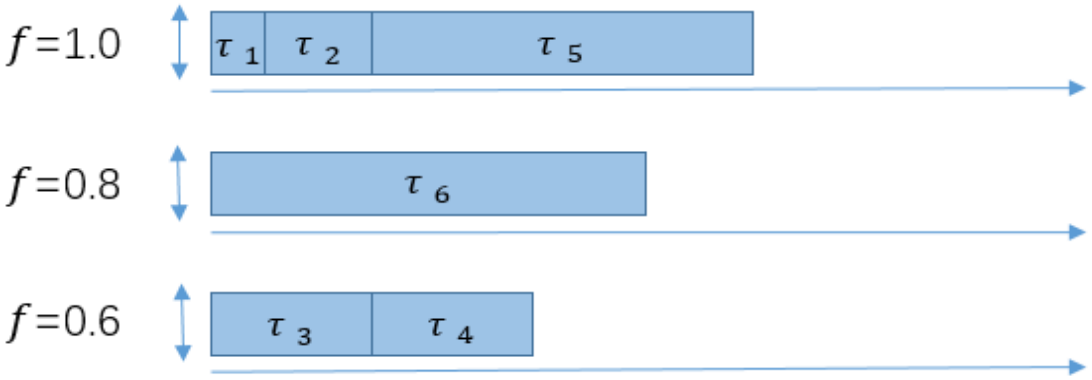
本节主要探究服务请求的分配对云提供商和用户的影响，主要体现在对云提供商收入和用户满意度的影响。本节首先假设某一时刻有如表 5.1 所示的服务请求集合，同时有频率分别为 1.0、0.8 和 0.6 的空闲服务器，则图 5.1(a)和图 5.1(b)分别代表两种不同的服务请求分配方案。

表 5.1 探究服务请求分配影响的服务请求集

| | 指令条数 | 服务价格 | 希望完成时间 | 低服务质量容忍度 | 截止时间 |
|---|---|---|---|---|---|
| 服务请求$\tau_1$ | 1 | 10 | 3 | 0 | 3 |
| 服务请求$\tau_2$ | 2 | 9 | 3 | 0.4 | 11.2 |
| 服务请求$\tau_3$ | 3 | 8 | 5 | 0.6 | 19.4 |
| 服务请求$\tau_4$ | 3 | 7 | 7 | 0.5 | 17.5 |
| 服务请求$\tau_5$ | 6 | 7 | 10 | 0.3 | 22.6 |
| 服务请求$\tau_6$ | 8 | 10 | 11 | 0.2 | 27 |




(a) 第一种请求分配方案




(b) 第二种请求分配方案

图 5.1 两种服务请求分配方案

如图 5.1 所示，第一种分配方案将服务请求$\tau_1$和$\tau_2$分配到频率为 1.0 的服务器上，将$\tau_3$和$\tau_4$分配到频率为 0.8 的服务器上，将$\tau_5$和$\tau_6$分配到频率为 0.6 的服务器上；而第二种分配方案则将$\tau_1$、$\tau_2$和$\tau_5$分配到频率为 1.0 的服务器，将$\tau_6$单独分配到频率为 0.8 的服务器上，将$\tau_3$和$\tau_4$分配到频率为 0.6 的服务器上。两种分配方案中，所有服务请求的完成时间、收入和用户满意度如表 5.2 所示。

表 5.2 两种服务请求分配方案结果

| 第一种请求分配方案结果 | | | | | | |
|---|---|---|---|---|---|---|
| | $\tau_1$ | $\tau_2$ | $\tau_3$ | $\tau_4$ | $\tau_5$ | $\tau_6$ |
| 完成时间 | 1 | 3 | 3.75 | 5 | 15 | 28.3 |
| 收入 | 10 | 18 | 24 | 21 | 25.3 | 0 |
| 满意度 | 10 | 10 | 10 | 10 | 6 | 0 |
| 第二种请求分配方案结果 | | | | | | |
| | $\tau_1$ | $\tau_2$ | $\tau_3$ | $\tau_4$ | $\tau_5$ | $\tau_6$ |
| 完成时间 | 1 | 3 | 5 | 10 | 9 | 10 |
| 收入 | 10 | 18 | 24 | 15 | 49 | 48 |
| 满意度 | 10 | 10 | 10 | 7 | 10 | 10 |

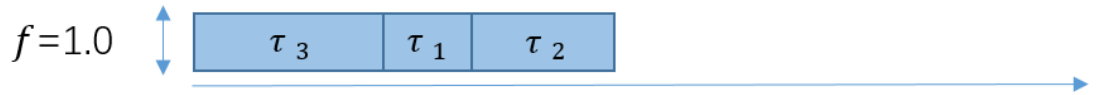
由表 5.2 可知，第一种服务请求分配方案一共可以得到 98.5 的总收入和 7.66 的用户平均满意度；第二种服务请求分配方案一共可以得到 164 的总收入和 9.5 的用户平均满意度。同样的服务请求集合和服务器，第二种分配方案可以将总收入和用户平均满意度分别提高 66.4%和 24%，而且第一种分配方案出现了服务请求超时的情况，这会严重影响用户的服务体验。由此可以看出，用户服务请求的分配无论是对云提供商的收入还是用户的满意度都有显著的影响。

### 5.1.2 服务请求执行顺序对云提供商和用户的影响

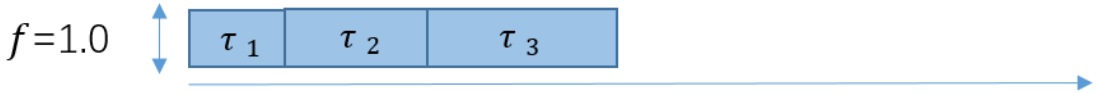
本节主要探究服务请求在服务器的执行顺序对云提供商和用户的影响。与上一节类似，本节首先假设某一时刻有如表 5.3 所示的服务请求集合，由于本节探讨的问题只需要一个服务器，因此假设只有一个频率为 1.0 的空闲服务器，对应的服务请求的数量也较少，则图 5.2(a)和图 5.2(b)分别代表两种不同的服务请求执行顺序。

表 5.3 探究服务请求执行顺序影响的服务请求集

| | 指令条数 | 服务价格 | 希望完成时间 | 低服务质量容忍度 | 截止时间 |
|---|---|---|---|---|---|
| 服务请求$\tau_1$ | 2 | 6 | 3 | 0 | 2 |
| 服务请求$\tau_2$ | 3 | 8 | 5 | 0.4 | 14.6 |
| 服务请求$\tau_3$ | 4 | 10 | 9 | 0.8 | 41 |




(a) 第一种服务请求执行顺序




(b) 第二种服务请求执行顺序

图 5.2 两种服务请求执行顺序

如图 5.2 所示，第一种服务请求执行顺序选择先执行服务请求$\tau_3$，再执行服务请求$\tau_1$，最后执行服务请求$\tau_2$；而第二种服务请求执行顺序则为先执行服务请求$\tau_1$，然后执行服务请求$\tau_2$，最后执行服务请求$\tau_3$。两种服务请求执行顺

序中，所有服务请求的完成时间、收入和用户满意度如表 5.4 所示。

表 5.4 两种服务请求执行顺序结果

| 第一种服务请求执行顺序结果 | | | |
|---|---|---|---|
| | $\tau_1$ | $\tau_2$ | $\tau_3$ |
| 完成时间 | 6 | 9 | 4 |
| 收入 | 0 | 14 | 40 |
| 满意度 | 0 | 6 | 10 |
| 第二种服务请求执行顺序结果 | | | |
| | $\tau_1$ | $\tau_2$ | $\tau_3$ |
| 完成时间 | 2 | 5 | 9 |
| 收入 | 12 | 24 | 40 |
| 满意度 | 10 | 10 | 10 |

由表 5.4 可知，第一种服务请求执行顺序一共可以得到 54 的总收入和 5.33 的用户平均满意度；第二种服务请求执行顺序一共可以得到 76 的总收入和 10 的用户平均满意度。只是改变了服务请求的执行顺序，第二种服务请求的执行顺序却分别提高了 40.7%和 87.6%的云提供商收入和用户满意度。因此，用户服务请求在同一个服务器的执行顺序也会影响到云提供商的收入和用户满意度。

## 5.2 基于价值评估和交叉熵的启发式利润优化算法研究

基于上述研究，本节提出了一个两阶段的启发式优化算法，用于个性用户服务请求的实时分配和调度。该启发式算法在第一个阶段使用交叉熵将个性用户服务请求分配到各个服务器上，然后在第二个阶段使用经济学的现值和机会成本对每个服务器上的服务请求进行价值评估，从而决定服务请求在每个服务器上的执行顺序。通过启发式优化算法的两个阶段，我们可以得到每个服务请求的服务器位置和在服务器中的执行顺序，从而实现用户满意度约束下云提供商的利润优化。

### 5.2.1 价值评估

由于本文的目标是优化云提供商利润，因此本节从利润的角度来评估每个服务请求的价值以决定服务请求的执行顺序，也就是利润的两个组成：收入和成本。

本节分别用经济学上的现值和机会成本来扩展服务请求的收入和成本的概念，以评估每个服务请求为云提供商带来的实际价值，然后就可以通过对所有服务请求带来的实际价值进行排序来决定服务请求的执行顺序。

**现值：**本节将收入的概念扩展以应对未来的风险。例如，给定具有相同单位收入和紧迫性的两个服务请求，云提供商最好先运行较短的服务请求，因为较长的服务请求对应于较长的运行时间，而较长的运行时间更有可能延迟后来到达的更有价值或更紧急的服务请求。换句话说，将资源投入长期工作可能会带来更高的风险，如果云提供商希望快速实现其收益或避免风险，他们甚至可以选择运行时间较少的低收入服务请求。在这里，本节使用经济学中的现值来扩展收入的定义。经济学中的现值是指未来现金在现在的价值，也称为贴现值。在本文中，服务请求$\tau_m$的现值定义为：

$$PValue_m = \frac{W_m * \ p_m}{(1 + discount_rate)} \tag{5.1}$$

其中$PValue_m$表示服务请求$\tau_m$考虑到未来风险的扩展收入，$W_m * \ p_m$表示服务请求$\tau_m$收入概念扩展前的收入，$discount_rate$表示贴现率，这是一个可调整的参数，参数越高意味着云提供商想要更快地实现回报，也表明未来的风险越高。

**机会成本：**决定用户服务请求的执行顺序最重要的是衡量每个服务请求的价值，如果仅仅从服务请求收入的角度来衡量显然是不够的。例如，服务请求$\boldsymbol{\tau_n}$比$\boldsymbol{\tau_m}$更紧急，即它的满意度和收入衰减得更快，那么即使它具有较低的单位收入，云提供商也可能选择先执行服务请求$\boldsymbol{\tau_n}$。更一般地说，即使服务请求具有高价值或高单位收入，云提供商也可以安全地推迟具有低衰减率的不太紧急的服务请求。因此，本节用经济学中的机会成本对服务请求的成本进行扩展以捕捉服务请求的紧迫性。机会成本在经济学上的定义为获得某种收入时放弃的其他收入，例如，一块土地种植了青菜就放弃了种植其他蔬菜的收入。对机会成本的分析，能够让云提供商选择成本最少的方案，也就是利润最多的方案，从而使云提供商有限的服务器资源得到最好的利用。

本节把机会成本定义为选择一个服务请求执行时，其他所有服务请求在该

服务请求执行过程中收入的衰减值之和。也就是说，如果云提供商要选择运行服务请求$\boldsymbol{\tau_m}$，那么必须放弃该服务器等候队列中的所有其他服务请求，这些被放弃的服务请求衰减的收入就是服务请求$\boldsymbol{\tau_m}$的机会成本。所以机会成本定义为：

$$OCost_m = \sum_{i=1;i!=m}^{M} a_i * \mathrm{MAX}(0, \frac{W_m}{f_n} - D_m^{des}) \tag{5.2}$$

其中$OCost_m$是服务请求$\tau_m$的机会成本，$M$是服务器等待队列中服务请求的数量，$a_i$表示服务请求$\tau_i$的收入衰减率，$\frac{W_m}{f_n}$表示服务请求$\tau_m$在服务器$S_n$上执行所需的运行时间，$W_m$为服务请求$\tau_m$的指令条数，$f_n$为服务器$S_n$的运行频率，$\frac{W_m}{f_n} - D_m^{des}$表示由于执行服务请求$\tau_m$而延误$\tau_i$的时间。当$\frac{W_m}{f_n} - D_m^{des}$为负数时，表示执行服务请求$\tau_m$并没有给服务请求$\tau_i$造成损失。

在对服务请求的收入和成本的概念进行扩展之后，每个服务请求能够为云服务商的实际价值可表示为：

$$SValue_m = PValue_m - OCost_m \tag{5.3}$$

在这里，$SValue_m$表示服务请求$\tau_m$的实际价值，它可以衡量每个服务请求为云提供商带来的价值，从而让云提供商能够优先执行带来更高利润的服务请求。$PValue_m$是服务请求$\tau_m$的现值，$OCost_m$是服务请求$\tau_m$的机会成本。值得注意的是$SValue_m$只是衡量服务请求价值的标准，不代表真实的利润。

结合上述概念，本节设计一个简单的服务请求排序算法来得到服务请求在每个服务器上的执行顺序，算法伪代码如 5.1 所示。

**算法 5.1 服务请求排序算法**

输入：　服务器集合$\{S_1, S_2, \ldots, S_N\}$；
分配到每个服务器上的服务请求集合$\{\boldsymbol{\Gamma_1}, \boldsymbol{\Gamma_2}, \ldots, \boldsymbol{\Gamma_N}\}$；

输出：　所有服务器上的服务请求执行顺序；

1: **for** *i* = *1* to *N* **do**
2: 计算服务器$S_i$的上服务请求集合$\boldsymbol{\Gamma_i}$大小，记为$M$；
/* 计算每个服务请求的实际价值 */
3: **for** *j* = *1* to *M* **do**
4: 根据公式（5.1）计算服务请求$\boldsymbol{\tau_j}$的现值；
5: 根据公式（5.2）计算服务请求$\boldsymbol{\tau_j}$的机会成本；
6: 将现值和机会成本带入公式（5.3）计算$\boldsymbol{\tau_j}$的实际价值；
7: **end for**
8: 根据服务请求的实际价值对服务请求集合$\boldsymbol{\Gamma_i}$进行排序；
9: 输出排序结果；
10: **end for**

算法 5.1 的输入是服务器集合$\{S_1, S_2, \ldots, S_N\}$和对应的服务请求集合$\{\boldsymbol{\Gamma_1}, \boldsymbol{\Gamma_2}, \ldots, \boldsymbol{\Gamma_N}\}$，输出是所有服务器上的服务请求执行顺序。该算法首先计算每个服务器上的服务请求个数（第 1-2 行），然后根据公式（5.1）（5.2）（5.3）计算所有服务请求给云提供商带来的实际价值（第 3-7 行），最后按照服务请求的实际价值对服务请求集合$\{\boldsymbol{\Gamma_1}, \boldsymbol{\Gamma_2}, \ldots, \boldsymbol{\Gamma_N}\}$进行排序并输出排序结果（第 8-10 行）。

### 5.2.2 交叉熵

本节介绍了交叉熵（Cross Entropy，CE）以及如何应用它来获得 NP-hard 优化问题的最优解。CE 方法的主要思想是将原始确定性优化问题转化为相关的随机问题，然后使用自适应采样算法来解决随机问题。在采样算法的每次迭代中，构造随机解的序列，让其收敛于最优解或近似最优解。

考虑以下优化问题，假设$\chi$是一组有限状态的集合，$S(x)$是在$\chi$上定义的实值函数，目标是找到$\chi$中$S(x)$的最大值$\varphi^*$和相应状态$x^*$，即：

$$\mathrm{S}(x^*) = \varphi^* = \max_{x\in\chi} \mathrm{S}(x) \tag{5.4}$$

为了求解(5.4)，优化问题(5.4)首先通过在集合$\chi$上定义一组概率密度函数$\{f(x,p), p \in U\}$来随机化，其中$p$是概率密度函数的参数。然后，该优化问题与以下估计问题相关联：

$$\lambda(\varphi) = P_u\{\mathrm{S}(X) \geq \varphi\} = E_u\left(I_{\{\mathrm{S}(X)\geq\varphi\}}\right) \tag{5.5}$$

其中，$\varphi$是一个参数，$\mathrm{X} = \{X_1, X_2, \ldots, X_Z\}$是由概率密度函数$\{f(x,u), u \in U\}$产生的$Z$个随机样本组成的样本向量。$P_u$表示实值函数$\mathrm{S}(X)$大于或等于$\varphi$的概率，$E_u$表示$I_{\{\mathbf{S}(\boldsymbol{X})\geq\varphi\}}$的数学期望，$I_{\{\mathbf{S}(\boldsymbol{X})\geq\varphi\}}$是指标函数，表示为：

$$I_{\{\mathrm{S}(X)\geq\varphi\}} \begin{cases} 1, \mathrm{S}(X) \geq \varphi \\ 0, \mathrm{S}(X) < \varphi \end{cases} \tag{5.6}$$

在将原始优化问题转换成（5.5）的估计问题后，CE 的主要思想是找到最小的$\varphi$使得$\lambda(\varphi)$接近 0，即实值函数$\mathrm{S}(X) \geq \varphi$的概率接近 0。实值函数$\mathrm{S}(X) \geq \varphi$的概率接近 0 表示$\mathrm{S}(X)$极大概率小于$\varphi$，而 CE 方法找到的是最小的$\varphi$，也就是说$\varphi$是实值函数$\mathrm{S}(X)$在$\chi$集合上的最小上界，即优化问题（5.4）的最优解。在上述思想中有两个问题需要解决：如何用已知的$\varphi$计算$\lambda(\varphi)$以及如何在给定当前$\varphi$的情况下获得更好的$\varphi$。接下来本节将说明如何解决这两个问题。

对于第一个问题，即如何用已知的$\varphi$计算$\lambda(\varphi)$，最简单直接的方法就是对解空间$\chi$进行穷举搜索来计算$\lambda(\varphi)$，但这明显是不合实际的。比较实际的方法是从解空间$\chi$生成一些样本通过蒙特卡洛模拟来估计$\lambda(\varphi)$，也就是使用概率密度函数$f(x,p)$生成$Z$个样本来估计$\lambda(\varphi)$，即：

$$\hat{\lambda} = \frac{1}{Z}\sum_{i=1}^{Z} I_{\{\mathrm{S}(\mathrm{X_i})\geq\varphi\}} \tag{5.7}$$

蒙特卡罗模拟在$\mathrm{S}(\mathrm{X_i}) \geq \varphi$为大概率事件时能够快速有效的估计$\lambda(\varphi)$，但当$\mathrm{S}(\mathrm{X_i}) \geq \varphi$为小概率事件时则需要大量样本来准确估计$\lambda(\varphi)$。在本文中，我们的目标是寻找$\mathrm{S}(X)$的最大值，当$\varphi$逐渐接近目标最大值时，$\mathrm{S}(\mathrm{X_i}) \geq \varphi$必然会成为

小概率的罕见事件，这会让蒙特卡罗模拟变得极为低效。因此，本节使用重要性采样对$\lambda(\varphi)$进行估计，即从解空间$\chi$中另一个不同的概率密度函数$k(x,u)$进行采样，而对原先的分布进行估计。重要性采样通过改变概率分布来以受控的方式增加罕见事件的数量，从而让采样点覆盖在对估计$\lambda(\varphi)$贡献更大的点。我们使用以下公式来计算$\lambda(\varphi)$的估计$\hat{\lambda}(\varphi)$：

$$\hat{\lambda}(\varphi)=\frac{1}{Z}\sum_{i=1}^{Z}I_{\{S(X_i)\geq\varphi\}}\frac{f(X_i,p)}{k(X_i,u)} \tag{5.8}$$

如果我们将概率密度函数$k^*(x,u)$定义为：

$$k^*(x,u)=\frac{I_{\{S(X_i)\geq\varphi\}}f(x,p)}{\lambda(\varphi)} \tag{5.9}$$

并将$k^*(x,u)$作为$k(x,u)$代入公式(5.8)，则：

$$\hat{\lambda}(\varphi)=\frac{1}{Z}\sum_{i=1}^{Z}I_{\{S(X_i)\geq\varphi\}}\frac{f(X_i,p)}{k^*(X_i,u)}=\lambda(\varphi) \tag{5.10}$$

即我们只需要找到$k^*(x,u)$就可以估计$\lambda(\varphi)$，但是$k^*(x,u)$的计算需要$\lambda(\varphi)$，而$\lambda(\varphi)$是未知的，这使得直接计算$k^*(x,u)$变得很复杂。因此，本节使用相对熵去寻找一个接近$k^*(x,u)$的概率分布。相对熵又被称为 Kullback-Leibler 散度，可以用来确定两个概率密度函数$k(x,u)$和$f(x,p)$之间的距离，相对熵的定义如下：

$$d(k||f)=k(x,u)\ln\frac{k(x,u)}{f(x,p)} \tag{5.11}$$

$$=\int k(x,u)\ln k(x,u)\,dx-\int k(x,u)\ln f(x,p)\,dx \tag{5.12}$$

只要将$k^*(x,u)$和$f(x,p)$之间的距离最小化，就可以认为$f(x,p)$是接近$k^*(x,u)$的概率分布，而$k^*(x,u)$是不变的，因此最小化公式（5.12）就相当于最大化其后半部分，这一部分被称为$k^*(x,u)$和$f(x,p)$的交叉熵，即：

$$\max_p\int k^*(x,u)\ln f(x,p)\,dx \tag{5.13}$$

将公式（5.9）带入，得到：

$$\max_{p} \int \frac{I_{\{S(X_i)\geq\varphi\}} f(x,p)}{\lambda(\varphi)} \ln f(x,p)\, dx \tag{5.14}$$

即：

$$\max_{p} E_u\left(I_{\{S(X)\geq\varphi\}}\right) \ln f(X,p)\, dx \tag{5.15}$$

因此，能够获得$k^*(x,u)$的最优$p$为：

$$p^* = \arg\max_{p} \frac{1}{Z} \sum_{i=1}^{Z} I_{\{S(X_i)\geq\varphi\}} \ln f(X_i,p) \tag{5.16}$$

结合上述步骤，我们通过$p^*$获得能够准确估计$\lambda(\varphi)$的概率分布$k^*(x,u)$，然后基于重要性采样通过公式(5.10)对$\lambda(\varphi)$进行计算。

对于如何在给定当前$\varphi$的情况下获得更好的$\varphi$这个问题，基于 CE 的方法通过迭代更新来得到一个更好的$\varphi$。在每次迭代中，基于 CE 的方法首先根据概率密度函数$f(x,p)$生成多个样本，然后计算每个样本对应方案的性能并根据性能对所有样本进行排序。在排完序后，挑选一定数量性能最好的样本用来更新下次迭代时的概率密度函数$f(x,p)$，同时从这一批样本中产生下一次迭代的$\varphi$。由于每次用性能最好的样本来获得下一次迭代的概率密度函数$f(x,p)$，所以能保证下一次迭代产生的样本的性能更好，即获得更好的$\varphi$。图 5.3 为一次迭代中概率密度函数$f(x,p)$的更新过程。假设$\varphi_t$为第$t$次迭代时的最大值$\varphi$，$f(x,p_t)$为第$t$次迭代时的概率密度函数，则具体的迭代过程如下：

- 更新$\varphi_t$：本节将$S(X)(X=\{X_1,X_2,\dots,X_Z\})$的第$(1-\rho)$分位数作为$\varphi_t$的值。具体来说，我们首先通过$f(x,p_{t-1})$产生 $Z$ 个随机样本$X_1,X_2,\dots,X_Z$，然后我们计算 Z 个随机样本相关联的函数值$S(X_1),S(X_2),\dots,S(X_Z)$，最后根据函数值的大小进行排序，得到$S_1,S_2,\dots,S_Z$。则排完序后的第$(1-\rho)$分位数为$\varphi_t$，即：

$$\varphi_t = S_{\lceil(1-\rho)*Z\rceil} \tag{5.17}$$

- 更新$p_t$：给定$\varphi_t$和$p_{t-1}$，我们可以通过解公式(5.16)得到下一个迭代的$p_t$，即：

$$p_t = arg\max_{p_t} \frac{1}{Z}\sum_{i=1}^{Z} I_{\{S(X_i)\geq\varphi_t\}} \ln f(X_i, p_t) \tag{5.18}$$

其中$X_i$和$S(X_i)$是通过$f(x, p_{t-1})$产生的样本和对应的函数值。

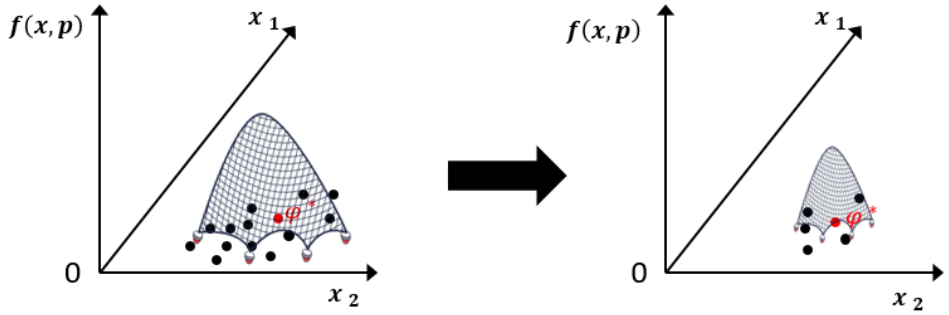



图 5.3 概率密度函数的更新过程

为了用 CE 确定获得最大利润的服务请求分配，在本文中，我们将样本$X$定义为$Z \times N$的矩阵，其中$Z$为生成的样本个数，$N$ 为服务请求的个数，则$X_{z,i}$=$j$表示第 $z$ 个样本中第 $i$ 个服务请求被分配到服务器 $j$ 上。此外，本节使用$N \times M$的概率矩阵 P 来生成样本，$P_{t,i,j}$表示第 $t$ 次迭代时服务请求 $i$ 被分配到服务器 $j$ 的概率。结合公式(5.18)，本节可以得到$P_{t,i,j}$的更新公式为：

$$P_{t,i,j} = \frac{\sum_{z=1}^{Z} I_{\{X_{z,i}=j\}} I_{\{S(X)\geq\varphi_t\}}}{\sum_{z=1}^{Z} I_{\{S(X_Z)\geq\varphi_t\}}} \tag{5.19}$$

其中，$S(X_Z)$表示样本$X_Z$使用公式（2.13）计算出来的云提供商的利润。

## 5.3 基于价值评估和交叉熵的启发式利润优化算法

基于上述研究，本节提出了一个基于价值评估和交叉熵的启发式利润优化算法，用于个性用户服务请求的实时分配和调度。该启发式算法在第一个阶段使用交叉熵将用户请求分配到各个服务器上，然后在第二个阶段使用经济学的

现值和机会成本对每个服务器上的服务请求进行价值评估，从而决定服务请求在每个服务器上的执行顺序。通过这两个阶段，云提供商可以得到所有服务请求的服务器位置和在服务器中的执行顺序，从而实现用户满意度约束下的利润优化。该算法的伪代码如算法 5.2 所示，这里值得注意的是算法 5.2 可以直接取代算法 4.1 中的求解 ILP 过程（第 9 行），其余部分与算法 4.1 类似，因此不再赘述。

**算法 5.2 基于价值评估和交叉熵的启发式利润优化算法**

输入: 用户服务请求集合 $\Gamma$，多服务器系统$\{S_1, S_2, \dots, S_n\}$;
输出: 云提供商利润，服务请求调度表;
/*初始化交叉熵方法的概率矩阵 */
1: **for** *n=1* **to** *N* **do**
2: **for** *m=1* **to** *M* **do**
3: $P_{0,n,m}$=*1/N;*
4: **end for**
5: **end for**
6: *t = 1;*
/* 基于价值评估和交叉熵生成解决方案 */
/* 当连续$k$次迭代结果相同或达到规定的迭代次数$t_{max}$时终止迭代 */
7: **while** $\varphi_{t-k} = \varphi_{t-k+1} = \dots = \varphi_t$ **or** $t = t_{max}$ **do**
8: 使用拉丁超立方体重要性抽样根据$P_{t-1}$生成 $Z$ 个样本，每个样本表示服务请求集合 $\Gamma$ 对服务器的分配;
9: 挑选 $K$ 个满足约束的可行样本$\{X_1, X_2, \dots, X_K\}$;
10: 使用算法 4.1 对分配到同一个服务器的服务请求进行排序;
11: 使用公式（2.13）计算每个样本产生的利润$S(X_1), \dots, S(X_K)$;
12: 使用公式（5.17）更新$\varphi_t$的值;
/* 根据更新后的$\varphi_t$更新概率矩阵$P_t$ */
13: **for** *n=1* **to** *N* **do**
14: **for** *m=1* **to** *M* **do**
15: 根据公式（5.19）和$\varphi_t$更新$P_{t,n,m}$的值;
16: **end for**
17: **end for**
18: *t=t+1*
19: **end while**
20: **return** $\varphi_t$, $X^*$;

算法 5.2 的输入是用户服务请求集合 $\Gamma$ 和多服务器系统$\{S_1, S_2, \dots, S_n\}$，输出是云提供商的利润和服务请求调度表。该算法首先对 CE 方法中的概率矩阵和迭代次数进行初始化（第 1-6 行）。初始化结束后，该算法基于价值评估和交叉

熵对服务请求调度队列中的服务请求进行调度（第 7-19 行）。在 CE 方法之前，该算法首先检查是否满足迭代结束条件，即连续$k$次迭代结果相同或达到规定的迭代次数$t_{max}$（第 7 行）。若满足迭代终止条件则终止迭代，否则该算法根据上一次迭代更新或初始化后的概率矩阵$\mathrm{P}_{t-1}$生成 $K$ 个满足约束的可行样本$\{X_1, X_2, \ldots, X_K\}$，这 $K$ 个可行样本指的是服务请求到服务器的分配（第 8-9 行）。然后，对这 $K$ 种分配方案使用算法 5.1 确定服务请求在服务器的执行顺序，并使用公式（2.13）计算排序后的方案产生的利润，将该利润作为最新的$\varphi_t$值（第 10-12 行）。接着，该算法根据更新后的$\varphi_t$和公式（4.31）更新概率矩阵$\mathrm{P}_t$并将$\mathrm{P}_t$作为下一次迭代的概率矩阵（第 13-19）。最后，该算法返回优化后的云提供商利润$\varphi_t$和对应的调度表$X^*$（第 20 行）。

## 5.4 本章小结

本章主要设计了启发式利润优化算法来克服基于 ILP 的调度方案的局限性。具体来说，本章首先讨论了服务请求分配和执行顺序对用户和云提供商的影响，然后使用经济学中的现值和机会成本来评估服务请求价值，使用交叉熵确定服务请求在多服务器系统中的分配，最后基于价值评估和交叉熵提出了两阶段的启发式利润优化算法。该启发式优化算法有两个阶段，第一个阶段使用交叉熵将个性用户的服务请求分配到各个服务器，第二个阶段使用经济学中的现值和机会成本对每个服务器中的服务请求进行价值评估，从而决定服务请求在服务器中的执行顺序。通过这两个阶段，该算法可以得到服务请求在服务器中的位置和执行顺序，从而实现用户满意度约束下的云提供商利润优化。

# 第六章 实验结果与分析

本章主要设计实验来验证用户满意度预测模型和用户满意度约束下的云提供商利润优化方案的有效性。对于用户满意度预测模型，主要验证该模型对用户满意度分数的预测准确率；对于用户满意度约束下的云提供商利润优化方案，主要验证了所提出方案在利润和算法运行时间上的优化效果，并与对比算法进行了对比。

## 6.1 实验设置

本文算法均运行在 Windows 系统下的 MatlabR2016b 版本，所使用的电脑配置为四核 2.50GHz Intel i7 处理器和 8GB DDR4 内存，所使用的开发语言主要为 Python、Java 和 Matlab。本文有两个实验：用户满意度预测模型实验和用户满意度约束下的云提供商利润优化实验，因此本节分别对这两个实验的设置进行介绍。用户满意度预测模型实验主要介绍了模型训练和模型验证的设置，用户满意度约束下的云提供商利润优化实验主要介绍了服务器、服务请求集合和实时电费的设置。

### 6.1.1 用户满意度预测模型实验设置

本节使用两组实验来验证用户满意度模型的准确性。第一组实验先通过问卷调查收集了 100 名参与者的个性评分和对于三十六种指定服务价格-服务质量的满意度，然后将这 100 名参与者的问卷数据分为五份，使用 5 折交叉验证来训练并验证用户满意度预测模型，即每次将 80 名参与者的问卷数据作为模型的训练数据，将其余 20 名参与者的问卷数据作为模型的验证数据，具体过程如图 6.1 所示。

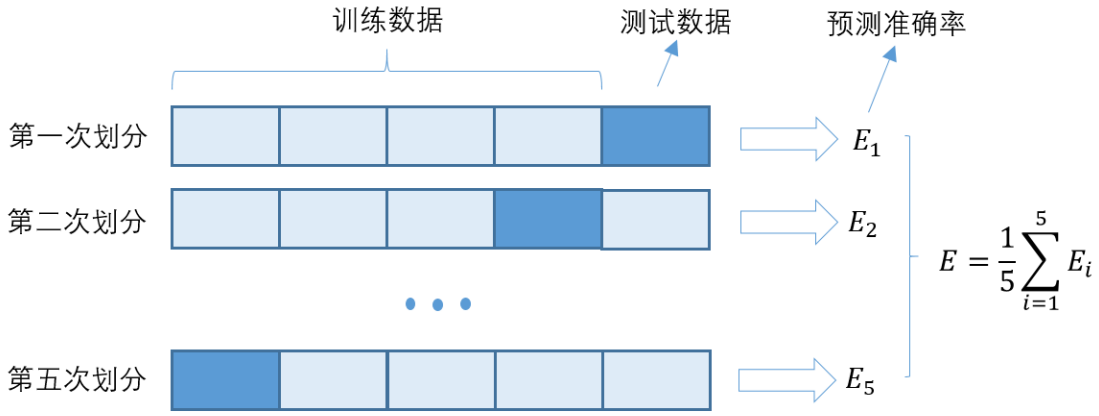



图 6.1 交叉验证过程

为了更好的验证所提出模型的有效性，本文招募了另外 100 名新参与者进行第二组实验，这些新参与者不参与模型的训练。同样，这 100 名参与者的个性评分和对于三十六种指定服务价格-服务质量的满意度通过问卷获得，然后基于所提出的满意度预测模型去预测这些参与者对指定服务价格-服务质量的满意度得分，最后将真实的满意度得分和预测的满意度得分进行对比分析。

### 6.1.2 用户满意度约束下的云提供商利润优化实验设置

本文所使用的服务器配置参数是基于文献[48]提出的，具体的服务器硬件配置如表 6.1 所示，每个服务器的配置从该表中随机选取。

表 6.1 服务器配置参数

| 等级 | 电压$v$(V) | 频率$f$(GHz) | 电容$C$(nF) |
|---|---|---|---|
| 1 | 0.85 | 0.8010 | 13.0 |
| 2 | 0.90 | 0.8291 | 12.0 |
| 3 | 0.95 | 0.8553 | 14.0 |
| 4 | 1.00 | 0.8797 | 15.0 |
| 5 | 1.05 | 0.9027 | 17.0 |
| 6 | 1.10 | 1.0000 | 16.0 |
| 7 | 1.15 | 1.0527 | 18.0 |
| 8 | 1.20 | 1.1236 | 16.0 |
| 9 | 1.25 | 1.1867 | 19.0 |
| 10 | 1.30 | 1.2500 | 18.0 |

由上文可知，用户的服务请求由五元组$\tau_m$：$\{W_m, p_m, D_m^{des}, \alpha_m, D_m\}$决定，本文将参与问卷调查的 200 名参与者随机分为十组，通过每组参与者的个性产生十

组服务请求集，其中每个服务请求的指令条数$W_m$在$[4*10^8, 6*10^9]$之间随机选择[48]，其他如服务价格$p_m$和希望完成时间$D_m^{des}$等属性通过所提出的用户满意度预测模型得到。此外，服务器运行产生的能耗和电费的换算使用实时电费计算形式[49]，其价格基于电力公司 Ameren 于 2014 年 1 月 1 日至 2014 年 1 月 31 日的实时价格，则电费计算如图 6.2 所示。




图 6.2 Ameren 实时电费[50]

## 6.2 实验结果与分析

### 6.2.1 用户满意度预测模型实验结果与分析

如上文所述，在第一组实验中，所有数据被分为五份以对用户满意度预测模型进行 5 折交叉验证。在每次交叉验证中，四份数据被用来作为训练集训练模型，一份数据作为测试集用来验证模型，然后根据预测结果和真实数据得到满意度分数预测的准确度。这样每份数据都能作为四次训练数据和一次测试数据，能够在有限的数据中提取到尽可能多的信息。本节将每次交叉验证中所有用户的预测满意度分数平均值作为预测值，将用户的真实满意度分数的平均值作为真实值，则所有迭代过程中预测值与真实值的比较如表 6.2 所示。

表 6.2 交叉验证结果

| 交叉验证组 | 预测值 | 真实值 | 误差 |
|---|---|---|---|
| 1 | 5.79 | 5.86 | -1.3% |
| 2 | 6.17 | 5.96 | 3.4% |
| 3 | 5.98 | 5.89 | 1.5% |
| 4 | 6.51 | 6.32 | 2.8% |
| 5 | 6.05 | 6.02 | 0.4% |

如表 6.2 所示，预测值与真实值的最大误差为 3.4%，最小误差为 0.4%，也就是说通过用户满意度预测模型和用户个性生成的服务请求能够很准确的捕捉用户满意度的变化。接下来，为了从整体上进一步的验证模型的有效性，本节将所有交叉验证组的预测结果和真实满意度分数进行整合，得到了如图 6.3 所示的混淆矩阵。

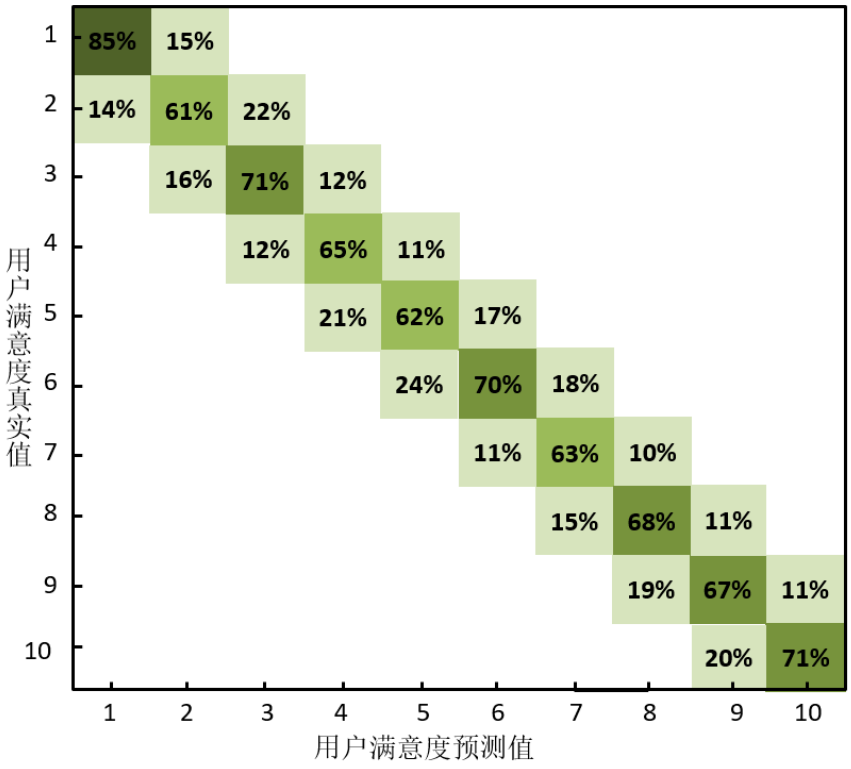



图 6.3 交叉验证中真实和预测的用户满意度得分的混淆矩阵

图 6.3 所示的混淆矩阵横坐标代表用户满意度分数的预测值，纵坐标表示用户真实的满意度分数（纵横坐标的取值范围都在 1 到 10 之间）。在该矩阵中，$(i, j)$ 即第 $i$ 行第 $j$ 列的元素表示真实满意度得分为 $i$ 且预测满意度得分为 $j$ 的概率，比如(1,1)=85%表示真实满意度得分为 1 且预测得分为 1 的概率为 85%，

即 85%的真实得分为 1 的案例被正确预测。（2，1）=15%意味着 15%真实满意度得分为 2 的案例被预测为 1。从该混淆矩阵可以看出，超过 68%的案例中所提出的模型能够准确预测出用户的满意度分数，超过 96%的案例中所提出的模型预测得分与真实得分差异在 1 以内。

在第二组实验中，为了验证所训练的模型是否对所有用户有效，100 名新参与者被分为十个组，每个组作为一个样本进行用户满意度预测模型的验证，验证结果如图 6.4 所示。

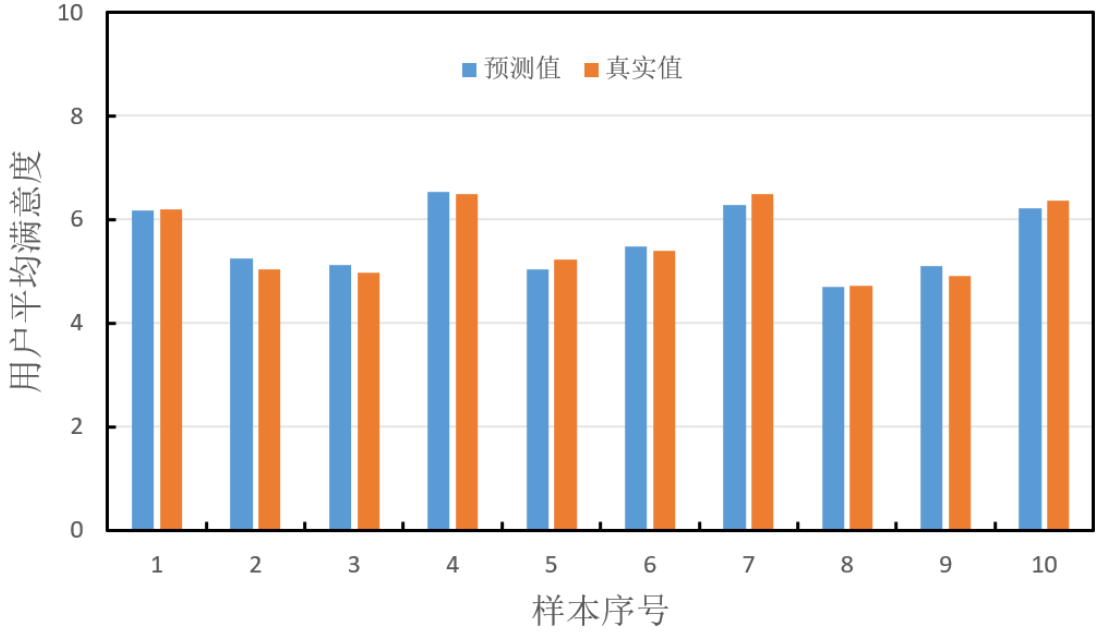



图 6.4 新参与者的测试结果

从图 6.4 可以看出，即使对于未参与模型训练的新参与者，所提出的预测模型的满意度预测值和用户的满意度真实值也几乎相等，这说明了所提出的满意度预测模型能够有效的预测所有用户的满意度分数。从图中可以看出，用户平均满意度分数在 4.5-6.5 之间，远远低于最高满意度分数 10 分，这是因为大多数用户不满意高服务价格和低服务质量的服务，因此给出了很低的满意度分数。同样的，用户对于低服务质量的容忍度不同导致了每组样本的平均满意度分数也不同。接下来，本节同样给出了所有样本中预测结果和真实满意度分数的混淆矩阵，如图 6.5 所示。

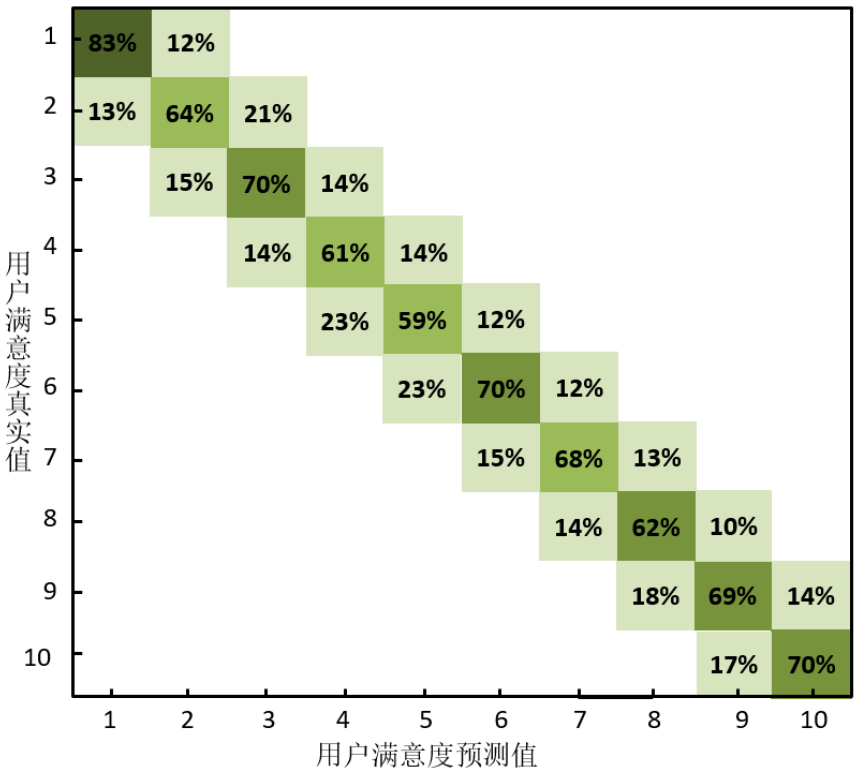



图 6.5 新参与者中真实和预测的用户满意度得分的混淆矩阵

从图 6.5 可以看出，在新参与者的满意度分数预测验证中，超过 67%的案例能够准确预测出用户的满意度，超过 91%的案例能够将预测误差控制在 1 以内。在新参与者中的预测准确率虽然略低于交叉验证中的预测准确率，但仍能够较为精准的预测出用户的满意度分数。

综上所述，本文所提出的用户满意度模型能够准确的预测出参与者对于指定服务价格-服务质量级别的满意度得分，将预测的用户平均满意度误差控制在 3.4%以内，并且超过 91%的预测案例误差不超过 1。

### 6.2.2 用户满意度约束下的云提供商利润优化实验结果与分析

由上文可知，本节将所有用户随机分为十组并根据每组用户的个性分别生成了十个服务请求集合。为了验证所提出的云提供商利润优化算法的有效性，本节将所提出的调度算法与 FIFS 算法、GA[51]和 CORA 算法[52]进行对比和分析，下面简单介绍下这些对比算法：

- FIFS（First In First Service）：先来先服务是 Hadoop 作业调度的默认调度方法，该方法将到来的用户服务请求放入一个等待队列中，当服务器空闲的

时候，从队列头部取出用户服务请求放入服务器执行。由于队列的特性，队列头部的服务请求是最先到达等候队列的服务请求，也就是说先到来的用户最先被服务，因此被称为先来先服务。

- GA（Genetic Algorithm）：遗传算法通过模拟自然进化过程来寻找最优解，通过编码将候选解抽象成染色体，使用适应度函数评价染色体的优劣，并使用繁殖和变异产生新的候选解，直到达到终止条件。
- CORA（Completion-time Optimal Resource Allocation）：该算法针对用户服务请求的完成时间进行了优化，通过将研究问题由整数规划转换为线性规划来获取最优解。

接下来，本节将在限制用户满意度的情况下，分别在利润和算法运行时间上对所提出的算法和对比算法进行实验和分析。

**云提供商利润：**云计算作为商业模式，利润是云提供商的最终目标，本节分别将用户满意度约束R设定为 6、7、8、9，即用户平均满意度需要大于设定的R值才为有效调度方案，然后对所提出算法和对比算法在云提供商利润优化上进行对比和分析。为了便于称呼，本节所提出的基于整数线性规划的利润优化方案简称为 ILP-Based（Integer Linear Programming Based）方案，基于价值评估和交叉熵的利润优化方案简称为 ECE-Based（Economics and Cross Entropy Based）方案，每个服务请求集的对比实验都会进行 1000 次，将 1000 次实验的平均值作为实验的最终结果，实验结果如图 6.6、6.7、6.8 和 6.9 所示。

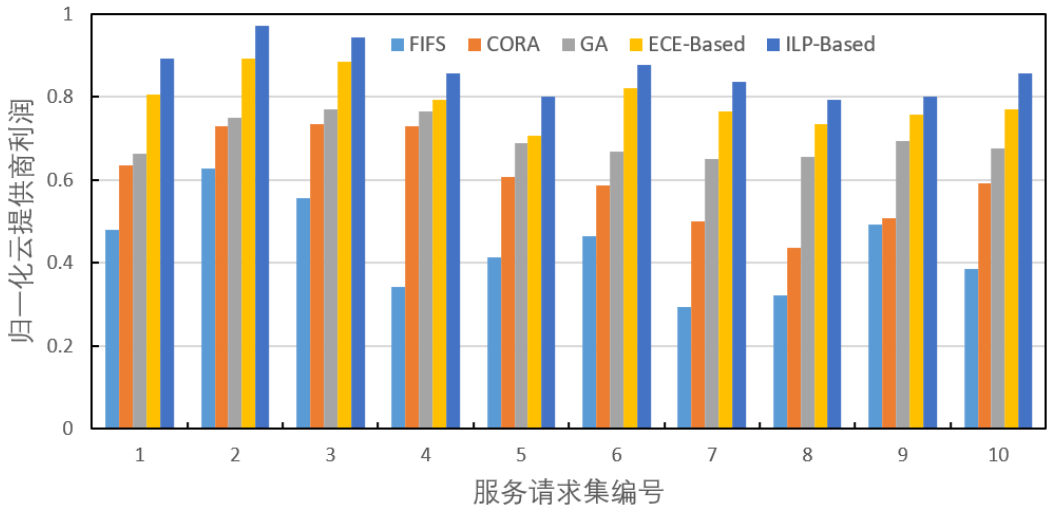



图 6.6 R = 6时五种算法云提供商利润优化对比

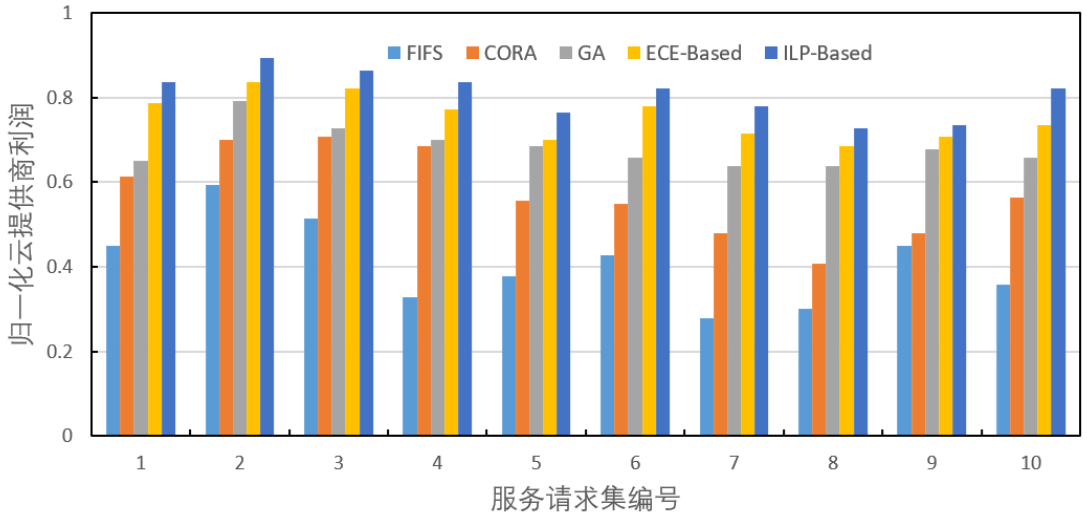



图 6.7 R = 7时五种算法云提供商利润优化对比

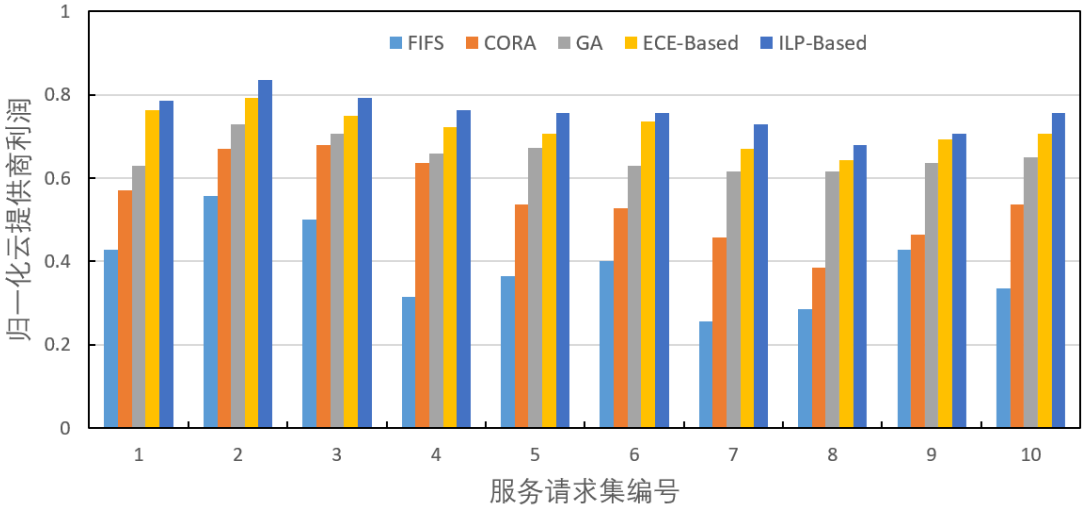



图 6.8 R = 8时五种算法云提供商利润优化对比

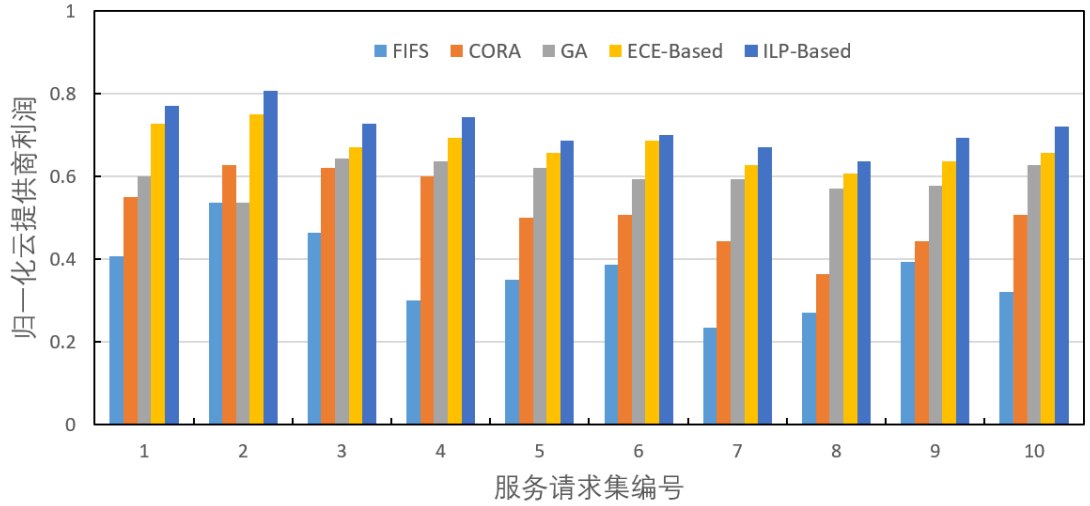



图 6.9 R = 9时五种算法云提供商利润优化对比

从图 6.6、6.7、6.8 和 6.9 可以看出，本文所提出的 ILP-Based 方案在多种不同的用户满意度约束下都能获得最高的利润，其次是本文提出的 ECE-Based 启发式算法，其他三个对比算法则产生较低的利润。本文所提出的两种算法之所以能够在云提供商利润优化上优于其他三个对比算法有两个主要原因：一个原因是所提出的算法能够捕捉并充分利用用户个性对满意度和利润的影响，这使得所提出的算法既不会错过高利润的服务请求也不会漏掉利润低但利润衰减很快的服务请求；第二个原因是所提出的算法是以利润最大化为目标函数，这使得所提出的算法能够在满意约束的情况下获得尽可能高的利润。而三个对比算法中，FIFS 按照用户服务请求到来时间来选择优先执行的服务请求，这种方法不需要额外的计算资源去得到服务请求的调度表，但是却很容易错过高利润和高紧急（即用户容忍度低，利润衰减快）的服务请求，这导致 FIFS 产生的利润远远低于其他对比算法；CORA 通过缩短所有服务请求的平均完成时间来提高利润，但是没有考虑到用户个性对服务请求属性的影响，一味的缩短所有用户的服务请求完成时间，不如适当的延迟低利润的服务请求完成时间，而将更多资源用来缩短高利润服务请求完成时间。但是该算法能够缩短平均服务请求完成时间，不容易错过紧急的服务请求从而避免了利润损失，因此优于 FIFS 算法；GA 虽然是以利润最大化为目标函数，但是没有充分利用用户服务请求本身的特征进行种群繁殖和变异，这使得算法容易早熟收敛而陷入局部最优解，在实验中多次出现了早熟现象，使得最终的平均利润变低，在利润优化上不如本文所提出的两种算法，但将利润作为适应度函数也使得高利润和高紧急的服务请求容易优先执行，因此优于 FIFS 方案和 CORA 方案。

此外，图 6.6、6.7、6.8 和 6.9 上我们可以看到同一种算法在同一个服务请求集合上的利润优化结果会随着满意度约束的提高而降低，这意味着用户满意度约束的提高会降低云提供商的利润。这是因为更高的用户满意度约束让云提供商必须优先处理紧急性高的服务请求从而避免满意度的降低，这导致紧急性低却高利润的服务请求会因此被错过而降低了利润。为了更直观的表现所提出算法在云提供商利润优化上的性能，本节在表 6.3 中总结了本文所提出的两种

算法与三种对比算法相比较在利润上的提升率。

表 6.3 所提出算法利润提升率

| ILP-Based 算法利润提升率 | | | | |
|---|---|---|---|---|
| | R=6 | R=7 | R=8 | R=9 |
| FIFS | 97.06% | 98.07% | 95.39% | 95.32% |
| CORA | 42.45% | 40.67% | 38.43% | 38.59% |
| GA | 23.58% | 18.30% | 15.59% | 19.29% |
| ECE-Based 算法利润提升率 | | | | |
| | R=6 | R=7 | R=8 | R=9 |
| FIFS | 81.24% | 84.76% | 85.61% | 83.24% |
| CORA | 31.01% | 31.22% | 31.50% | 30.01% |
| GA | 13.66% | 10.35% | 9.80% | 11.90% |

从表 6.3 中可以看出，本文所提出的 ILP-Based 算法与 FIFS 算法、CORA 算法和 GA 相比，在用户满意度约束为 6 时，分别提高了云提供商 97.06%、42.45%和 23.58%的利润；在用户满意度约束为 7 时，分别提高了云提供商 98.07%、40.67%和 18.30%的利润；在用户满意度约束为 8 时，分别提高了云提供商 95.39%、38.43%和 15.59%的利润；在用户满意度约束为 9 时，分别提高了云提供商 95.32%、38.59%和 19.29%的利润。

本文所提出的启发式 ECE-Based 算法与 FIFS 算法、CORA 算法和 GA 相比，在用户满意度约束为 6 时，分别提高了云提供商 81.24%、31.01%和 13.66%的利润；在用户满意度约束为 7 时，分别提高了云提供商 84.76%、31.22%和 10.35%的利润；在用户满意度约束为 8 时，分别提高了云提供商 85.61%、31.50%和 9.80%的利润；在用户满意度约束为 9 时，分别提高了云提供商 83.24%、30.01%和 11.90%的利润。

由此可以看出，本文所提出的 ILP-Based 算法能够获得用户满意度约束下的利润优化问题最优解，对比其他算法在相同的满意度约束下有很大的利润提升，而所提出的启发式 ECE-Based 算法虽然只能得到研究问题的近似最优解，但是在利润优化方面仍然优于三种对比算法。

**算法运行时间：**为了更全面的比较所提出的算法和对比算法在运行时间上的优劣，本节首先给出各个算法在服务请求增多的情况下运行时间的变化，然后给出所有算法在多次实验过程中运行时间的平均值。这里值得注意的是，

FIFS 算法使用简单的先来先服务方法决定服务请求调度结果，没有计算调度表的过程，因此算法运行时间接近于 0，不参与算法运行时间的比较。

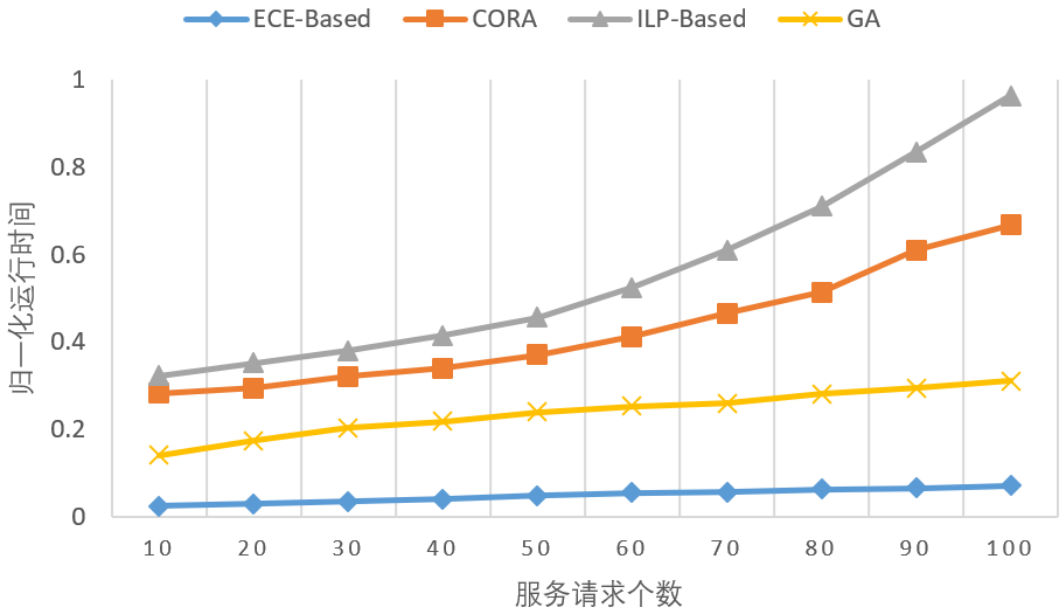



图 6.10 四种算法运行时间比较

从图 6.10 可以看出，ILP-Based 算法和 CORA 算法的运行时间远高于 GA 和 ECE-Based 算法，并且随着服务请求个数的增加，算法运行时间的增加也越加迅速。这是因为 ILP-Based 算法和 CORA 算法都是基于线性规划设计的算法，这导致他们的算法复杂度高于一般的近似算法，而 CORA 算法中约束条件和目标函数较为简单，因此算法运行时间优于本文所提出的 ILP-Based 算法。在近似算法 GA 和 ECE-Based 算法中，所提出的 ECE-Based 算法不但使用收敛更快的交叉熵方法，而且用了经济学中的现值和机会成本进行了优化，因此在运行时间上优于收敛速度较慢的 GA。下面本节将每个满意度约束下的四种算法运行 1000 次，并计算本文所提算法在平均运行时间上的提速比，具体实验结果见表 6.4。

表 6.4 四种算法平均运行时间

| | ECE-Based | ILP-Based | GA | CORA | Min. Impro | Max. Impro |
|---|---|---|---|---|---|---|
| R=6 | 1 | 21.09 | 5.87 | 18.75 | 5.87x | 21.09x |
| R=7 | 1.25 | 28.55 | 7.31 | 22.11 | 5.84x | 22.84x |
| R=8 | 1.41 | 26.19 | 8.49 | 19.09 | 6.02x | 18.57x |
| R=9 | 1.73 | 23.92 | 10.14 | 21.98 | 5.86x | 13.82x |

为了便以比较，表 6.4 中的平均运行时间以 ECE-Based 算法在用户满意度约束为 6 时的平均运行时间为基准进行了处理。从表 6.4 可以看出，本文所提出的启发式 ECE-Based 算法在平均运行时间上优于其他算法，其中最大的加速比达到 22.84 倍，最小的加速比也达到了 5.87 倍。此外，随着用户满意度约束值的提高，ECE-Based 算法和 GA 在运行时间上有不同程度的增加，这主要是因为约束条件的提高使得算法有效解变少，减少了这两种算法产生有效解的概率，从而加大了算法平均运行时间。

综合云提供商利润和算法平均运行时间的实验结果，本文所提出的 ILP-Based 算法虽然平均运行时间较久，但是能够实现用户满意度约束下的云提供商利润最大化，提供远高于对比算法的利润。而本文提出的启发式 ECE-Based 算法通过使用价值评估和交叉熵对算法运行时间进行了优化，不但能够实现仅次于最优解的云提供商利润，还能够有效的减少云提供商的计算资源消耗和调度时间，并且随着问题规模的增加依然能够起到大幅度缩短算法运行时间的效果。

## 6.3 本章小结

本章主要对用户满意度预测模型和用户满意度约束下的云提供商利润优化方案进行验证和结果分析。首先，本章介绍了这两个方案的实验设置，主要是用户满意度预测模型中模型训练和模型验证的设置，以及用户满意度约束下的云提供商利润优化实验中服务器、服务请求集合和实时电费的设置。然后，本章通过实验验证和分析这两个方案的有效性。对于用户满意度预测模型，主要验证用户满意度预测的准确性，在交叉验证的五组实验中该模型的预测误差最大为 3.8%，在新招募的参与者实验中超过 91%的案例预测值和真实值误差在 1 以内；对于用户满意度约束下的云提供商利润优化方案，主要验证算法在不同用户满意度约束下实现的云提供商利润和算法运行时间，并与其他三种算法进行对比分析。在实现利润上，本文所提出的 ILP-Based 算法相对于三种对比算法能够实现高达 97.06%、42.45%和 23.58%的利润提升，所提出的启发式 ECE-

Based 算法能够实现高达 84.76%、31.22%和 10.35%的利润提升。在算法运行时间上，所提出的启发式 ECE-Based 相对于其他算法能够实现最高 22.84 倍，最低 5.87 倍的加速比。

# 第七章 总结与展望

## 7.1 本文总结

本文的目的是设计调度算法决定个性用户的服务请求在多服务器系统上的分配以及执行顺序，从而实现云提供商利润的最大化，同时保证个性用户的满意度。实验表明，在不同的用户满意度约束下，本文所提出的算法能够有效的提升云提供商的利润。本文的研究工作总结如下：

本文提出了用户个性的概念，即用户对服务价格和服务质量有不同的偏好，将用户满意度模型建模为用户个性、服务价格和服务质量的函数以准确捕捉不同个性用户对指定服务价格-服务质量层级的满意度。接着，本文通过大五人格评分和问卷调查获取用户的个性评分，基于统计模型获取用户个性评分与用户偏好的关系，得到了用户满意度模型中各项参数的值。通过用户满意度模型，本文能够准确预测用户对于指定服务的满意度，从而使用合理的调度提高所有用户的平均满意度。

基于用户满意度模型，本文使用整数线性规划（ILP）对研究问题进行了建模和求解，并提出了基于 ILP 的云提供商利润优化方案。该方案虽然能够得到研究问题的最优解，但有着运行时间过长的局限性。因此，本文基于价值评估和交叉熵设计了启发式利润优化算法来克服基于 ILP 的调度方案的局限性。该启发式优化算法有两个步骤，第一步使用交叉熵将用户服务请求分配到各个服务器，然后通过经济学中的现值和机会成本对服务器中的服务请求进行价值评估，以决定服务请求在服务器中的执行顺序。通过这两个步骤，该算法可以得到服务请求在多服务器系统中的位置和执行顺序，从而实现用户满意度约束下的云提供商利润优化。

本文通过大量实验证明了所提出方案的有效性。所提出的用户满意度预测模型能够在超过 91%的案例中误差不超过 1，所提出的利润优化方案相比于 FIFS 算法、CORA 算法和 GA 三种对比算法最低有 81.24%、30.01%、9.8%的利润提

升，且最低有 5.87 倍的提速比。

## 7.2 对未来的展望

尽管本文设计了能够得到最优解的基于 ILP 的利润优化算法，也设计了运行速度较快能够得到近似解的启发式利润优化算法，但本文研究工作还有一些不足之处。因此，本文还可以从以下方面进行改进：

1. 对于服务器模型，本文只考虑了多核异构服务器，没有考虑服务器的动态电压频率调节技术（Dynamic Voltage and Frequency Scaling，DVFS），即每个服务器不但有不同结构的处理器核心，每个处理器核心还可以调节电压与频率。因此，未来我们将对服务器模型进一步完善，增加服务器的 DVFS 功能。
2. 对于优化目标，本文优化了所有用户的总满意度和云提供商的利润，没有考虑用户之间的公平性，这可能导致一些用户满意度过低。因此，我们未来考虑加入用户公平性的约束，缩小用户之间的满意度差距。
3. 本文假设用户提交的服务请求都是独立无依赖的，而实际上有些用户的服务请求可能是相互依赖的。因此，在未来的研究工作中，我们考虑加入有依赖关系的服务请求流来完善研究工作。

# 参考文献

[1] Hwang K, Dongarra J, et al. Distributed and Cloud Computing[M]. San Mateo, CA, USA: Elsevier/Morgan Kaufmann, 2012.

[2] Cao J, Hwang K, Li K, et al. Optimal multiserver configuration for profit maximization in cloud computing[J]. ieee transactions on parallel and distributed systems, 2013, 24(6): 1087-1096.

[3] Fox A, Griffith R, Joseph A, et al. Above the clouds: A berkeley view of cloud computing[J]. Dept. Electrical Eng. and Comput. Sciences, University of California, Berkeley, Rep. UCB/EECS, 2009, 28(13): 2009.

[4] Gartner. https://www.gartner.com.

[5] Lee Y C, Wang C, Zomaya A Y, et al. Profit-driven service request scheduling in clouds[C]//2010 10th IEEE/ACM International Conference on Cluster, Cloud and Grid Computing. IEEE, 2010: 15-24.

[6] Chen J, Wang C, Zhou B B, et al. Tradeoffs between profit and customer satisfaction for service provisioning in the cloud[C]//Proceedings of the 20th international symposium on High performance distributed computing. ACM, 2011: 229-238.

[7] Chun Y H. Optimal pricing and ordering policies for perishable commodities[J]. European Journal of Operational Research, 2003, 144(1): 68-82.

[8] Ghamkhari M, Mohsenian-Rad H. Energy and performance management of green data centers: A profit maximization approach[J]. IEEE Transactions on Smart Grid, 2013, 4(2): 1017-1025.

[9] Lee Y C, Wang C, Zomaya A Y, et al. Profit-driven scheduling for cloud services with data access awareness[J]. Journal of Parallel and Distributed Computing, 2012, 72(4): 591-602.

[10] Zhang Q, Zhu Q, Boutaba R. Dynamic resource allocation for spot markets in cloud computing environments[C]//2011 Fourth IEEE International Conference on Utility

and Cloud Computing. IEEE, 2011: 178-185.

[11] Macías M, Guitart J. A genetic model for pricing in cloud computing markets[C]//Proceedings of the 2011 ACM Symposium on Applied Computing. ACM, 2011: 113-118.

[12] Xu H, Li B. Dynamic cloud pricing for revenue maximization[J]. IEEE Transactions on Cloud Computing, 2013, 1(2): 158-171.

[13] Zhao J, Li H, Wu C, et al. Dynamic pricing and profit maximization for the cloud with geo-distributed data centers[C]//IEEE INFOCOM 2014-IEEE Conference on Computer Communications. IEEE, 2014: 118-126.

[14] Zhang X, Wu C, Li Z, et al. A Truthful Optimal Mechanism for On-demand Cloud Resource Provisioning[J]. IEEE Transactions on Cloud Computing, 2018.

[15] Zhang H, Jiang H, Li B, et al. A framework for truthful online auctions in cloud computing with heterogeneous user demands[J]. IEEE Transactions on Computers, 2016, 65(3): 805-818.

[16] Wang Q, Ren K, Meng X. When cloud meets ebay: Towards effective pricing for cloud computing[C]//2012 Proceedings IEEE INFOCOM. IEEE, 2012: 936-944.

[17] Mei J, Li K, Hu J, et al. Energy-aware preemptive scheduling algorithm for sporadic tasks on DVS platform[J]. Microprocessors and Microsystems, 2013, 37(1): 99-112.

[18] Goiri I, Julia F, Fito J, et al. Supporting cpubased guarantees in cloud slas via resource-level qos metrics[J]. Future Generation Computer Systems, 2012, vol. 28, no. 8, pp. 1295-1302.

[19] Mattess M, Vecchiola C, Buyya R. Managing peak loads by leasing cloud infrastructure services from a spot market[C]//2010 IEEE 12th International Conference on High Performance Computing and Communications (HPCC). IEEE, 2010: 180-188.

[20] Chen J, Wang C, Zhou B B, et al. Tradeoffs between profit and customer satisfaction for service provisioning in the cloud[C]//Proceedings of the 20th international symposium on High performance distributed computing. ACM, 2011: 229-238.

[21] Mei J, Li K, Ouyang A, et al. A profit maximization scheme with guaranteed quality of service in cloud computing[J]. IEEE Transactions on Computers, 2015, 64(11): 3064-3078.

[22] Rochwerger B, Breitgand D, Epstein A, et al. Reservoir-when one cloud is not enough[J]. Computer, 2011, 44(3): 44-51.

[23] Rochwerger B, Breitgand D, Levy E, et al. The reservoir model and architecture for open federated cloud computing[J]. IBM Journal of Research and Development, 2009, 53(4): 4: 1-4: 11.

[24] Goiri Í, Guitart J, Torres J. Economic model of a cloud provider operating in a federated cloud[J]. Information Systems Frontiers, 2012, 14(4): 827-843.

[25] Le K, Bianchini R, Zhang J, et al. Reducing electricity cost through virtual machine placement in high performance computing clouds[C]//Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis. ACM, 2011: 22.

[26] Mashayekhy L, Nejad M M, Grosu D. Cloud federations in the sky: Formation game and mechanism[J]. IEEE Transactions on Cloud Computing, 2015, 3(1): 14-27.

[27] Samaan N. A novel economic sharing model in a federation of selfish cloud providers[J]. IEEE Transactions on Parallel and Distributed Systems, 2014, 25(1): 12-21.

[28] Li X, Wu J, Tang S, et al. Let's stay together: Towards traffic aware virtual machine placement in data centers[C]//IEEE INFOCOM 2014-IEEE conference on computer communications. IEEE, 2014: 1842-1850.

[29] Cui L, Cziva R, Tso F P, et al. Synergistic policy and virtual machine consolidation in cloud data centers[C]//IEEE INFOCOM 2016-The 35th Annual IEEE International Conference on Computer Communications. IEEE, 2016: 1-9.

[30] Jiang J W, Lan T, Ha S, et al. Joint VM placement and routing for data center traffic engineering[C]//2012 Proceedings IEEE INFOCOM. IEEE, 2012: 2876-2880.

[31] Guo Y, Stolyar A L, Walid A. Shadow-routing based dynamic algorithms for virtual

machine placement in a network cloud[C]//2013 Proceedings IEEE INFOCOM. IEEE, 2013: 620-628.

[32] Lee Y C, Wang C, Zomaya A Y, et al. Profit-driven scheduling for cloud services with data access awareness[J]. Journal of Parallel and Distributed Computing, 2012, 72(4): 591-602.

[33] He Q, Zhu X, Li D, et al. Cost-effective Big Data Mining in the Cloud: A Case Study with K-means[C]//2017 IEEE 10th International Conference on Cloud Computing (CLOUD). IEEE, 2017: 74-81.

[34] Chen C, Wang W, Li B. Performance-Aware Fair Scheduling: Exploiting Demand Elasticity of Data Analytics Jobs[C]//IEEE INFOCOM 2018-IEEE Conference on Computer Communications. IEEE, 2018: 504-512.

[35] Khazaei H, Misic J, Misic V B. Performance analysis of cloud computing centers using m/g/m/m+ r queuing systems[J]. IEEE Transactions on parallel and distributed systems, 2012, 23(5): 936-943.

[36] Cao J, Li K, Stojmenovic I. Optimal power allocation and load distribution for multiple heterogeneous multicore server processors across clouds and data centers[J]. IEEE Transactions on Computers, 2014, 63(1): 45-58.

[37] Condor. (2014) [Online]. Available: http://www.cs.wisc.edu/condor.

[38] Thain D, Tannenbaum T, Livny M. Distributed computing in practice: the Condor experience[J]. Concurrency and computation: practice and experience, 2005, 17(2-4): 323-356.

[39] Durkee D. Why cloud computing will never be free[J]. Communications of the ACM, 2010, 53(5): 62-69.

[40] Zhang Y, Parikh D, Sankaranarayanan K, et al. Hotleakage: A temperature-aware model of subthreshold and gate leakage for architects[J]. University of Virginia Dept of Computer Science Tech Report CS-2003, 2003, 5.

[41] Chandrakasan A P, Sheng S, Brodersen R W. Low-power CMOS digital design[J].

IEICE Transactions on Electronics, 1992, 75(4): 371-382.

[42] Zhai B, Blaauw D, Sylvester D, et al. Theoretical and practical limits of dynamic voltage scaling[C]//Proceedings of the 41st annual Design Automation Conference. ACM, 2004: 868-873.

[43] Rotem E, Naveh A, Moffie M, et al. Analysis of thermal monitor features of the Intel Pentium M processor[C]//TACS Workshop at ISCA-31. 2004.

[44] Corsini R J, Ozaki B D. Encyclopedia of psychology[M]. New York: Wiley, 1994.

[45] McCrae R R, John O P. An introduction to the five-factor model and its applications[J]. Journal of personality, 1992, 60(2): 175-215.

[46] Gosling S D, Rentfrow P J, Swann Jr W B. A very brief measure of the Big-Five personality domains[J]. Journal of Research in personality, 2003, 37(6): 504-528.

[47] Lin H, Bilmes J. Multi-document summarization via budgeted maximization of submodular functions[C]//Human Language Technologies: The 2010 Annual Conference of the North American Chapter of the Association for Computational Linguistics. 2010: 912-920.

[48] Zhou J, Wei T, Chen M, et al. Thermal-aware task scheduling for energy minimization in heterogeneous real-time MPSoC systems[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2016, 35(8): 1269-1282.

[49] Zhan Y, Ghamkhari M, Xu D, et al. Extending demand response to tenants in cloud data centers via non-intrusive workload flexibility pricing[J]. IEEE Transactions on Smart Grid, 2018, 9(4): 3235-3246.

[50] Real Time Prices. https:// www2.ameren.com/RetailEnergy/RealTimePrices.

[51] Zhao C, Zhang S, Liu Q, et al. Independent tasks scheduling based on genetic algorithm in cloud computing[C]//2009 5th International Conference on Wireless Communications, Networking and Mobile Computing. IEEE, 2009: 1-4.

[52] Huang Z, Balasubramanian B, Wang M, et al. Need for speed: Cora scheduler for optimizing completion-times in the cloud[C]//2015 IEEE Conference on Computer

Communications (INFOCOM). IEEE, 2015: 891-899.

# 科研成果

[1] Guo Xu, Tongquan Wei, Junlong Zhou and Mingsong Chen. Leveraging User Heterogeneities to Maximize Profits in the Cloud. The 24th IEEE International Conference on Parallel and Distributed Systems, 2018.(CCF 推荐的 C 类会议) (第一作者)

[2] Kun Cao, Guo Xu, Junlong Zhou, Tongquan Wei, Mingsong Chen and Shiyan Hu. QoS-adaptive approximate real-time computation for mobility-aware IoT lifetime optimization. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2018. (CCF 推荐的 A 类期刊)(第二作者)

# 致谢

时光荏苒，岁月如梭，随着学位论文的完成，也预示着我的三年研究生生涯已接近尾声。经过三年的学习积累，我不但对云计算研究领域有了深入的了解，完成了专业知识的学习，实现了职业愿景，还结识了许多位可敬的老师和可爱的同学，锻炼了教学能力和交际能力，收获颇丰。这些都将是我此生最宝贵的财富。

感谢我最尊敬的导师魏同权老师。在我攻读硕士学位期间，魏老师严谨细致的治学之风和谦逊正直的为人处世之风，以及追求卓越的科研态度深深影响着我。得益于魏老师的悉心教导和言传身教，我才能顺利地完成学业并取得一定的科研成果。魏老师是我科研工作上的楷模，作为您的学生，我深感荣幸。在此，谨向魏老师表示衷心的感谢和崇高的敬意！

感谢 604 实验室的每一位同学：曹坤师兄以远超同龄博士的水平向我们展示了一个优秀博士生的风采，同时感谢曹博士对我论文的帮助；感谢女友王志红在我写作论文期间给我的帮助，帮我指出错误和语言表达问题，同时也感谢她生活上带给我的快乐，在我情绪低落时给我的鼓励；还感谢曾经一起学习的小伙伴们，我从你们身上学会了很多。

特别感谢我的父母和家人，不管发生什么事情，他们总给我最大的支持，是我坚强的后盾。祝他们身体健康，幸福开心。我将在今后的工作、学习中加倍努力，以期能够用自己的技能和头脑成为一个对社会有用的人。

最后，还特别感谢本文的各位评审老师，感谢你们能在百忙之中抽出宝贵的时间来审阅我的毕业论文。由于本人学术水平有限，文中缺陷在所难免，还望各位专家老师批评指正。